Institut Supérieur d'Informatique de Mahdia (ISIMa)

# IoT Appli et Plates formes

Des fondamentaux du capteur jusqu'à l'Edge AI et au TinyML — architecture, protocoles, cloud AWS et intelligence embarquée.

**Dr. Ahmed Bannour, MA en Télécommunications**

LCS3-GLSI

PLAN DU COURS

# Plan du cours — 9 modules

**01** **Fondamentaux de l'IoT**

Capteurs, actionneurs, dispositifs

**02** **Réseaux IoT**

Protocoles, topologies, technologies sans fil

**03** **MQTT**

Publish/Subscribe, QoS, sécurité

**04** **Cloud & Edge Computing**

Architecture multi-couches

**05** **Dispositif AIoT virtuel**

Simulateur Python, MQTT, MongoDB

**06** **Cloud AWS pour l'IoT**

IoT Core, DynamoDB, Kinesis, Batch

**07** **IA & Machine Learning**

Supervisé, non supervisé, évaluation

**08** **Edge AI & TinyML**

Matériel, optimisation de modèles

**09** **Études de cas & synthèse**

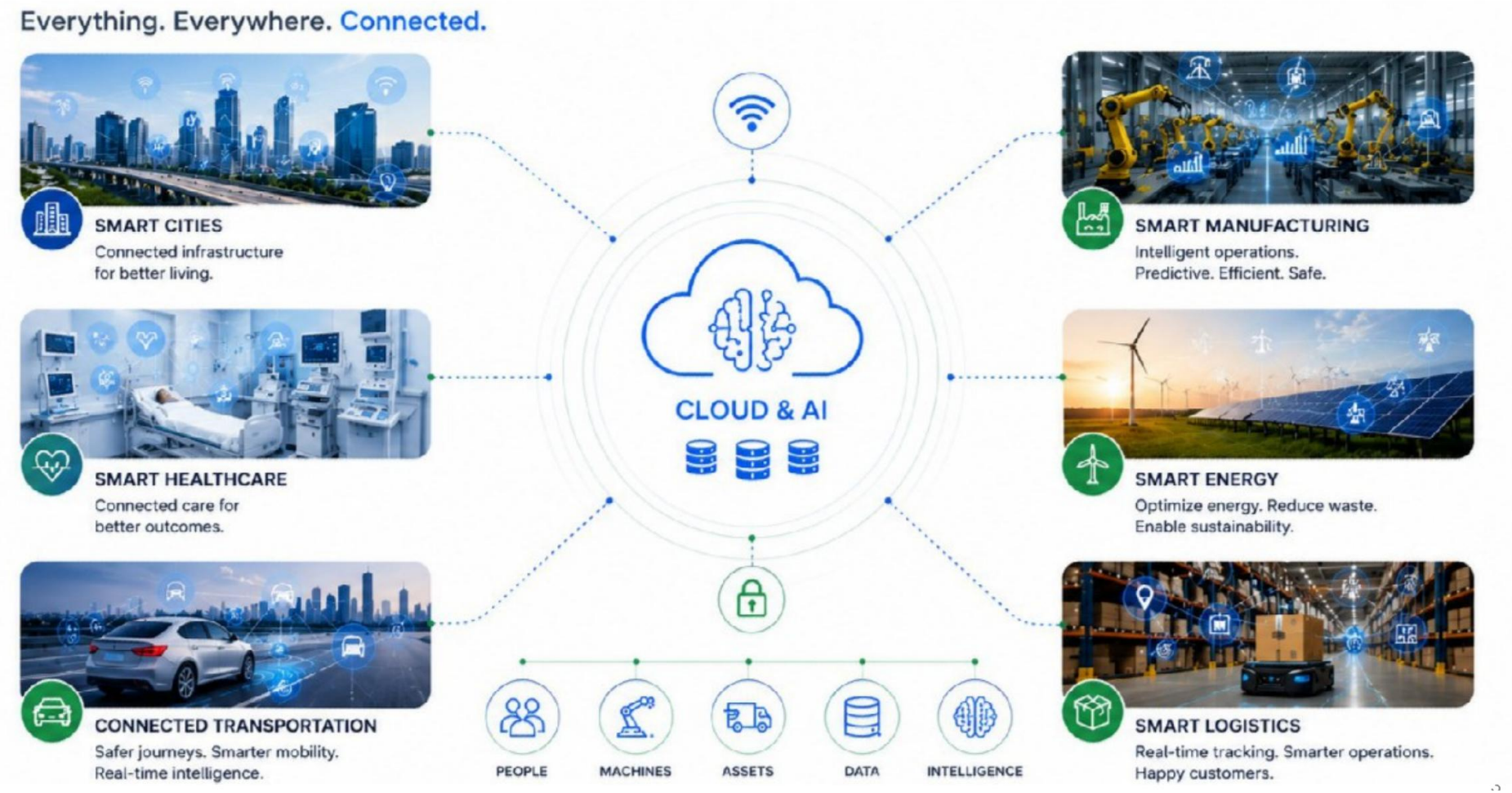
Cas réels, architecture complète

MODULE 1

# Fondamentaux de l'IoT

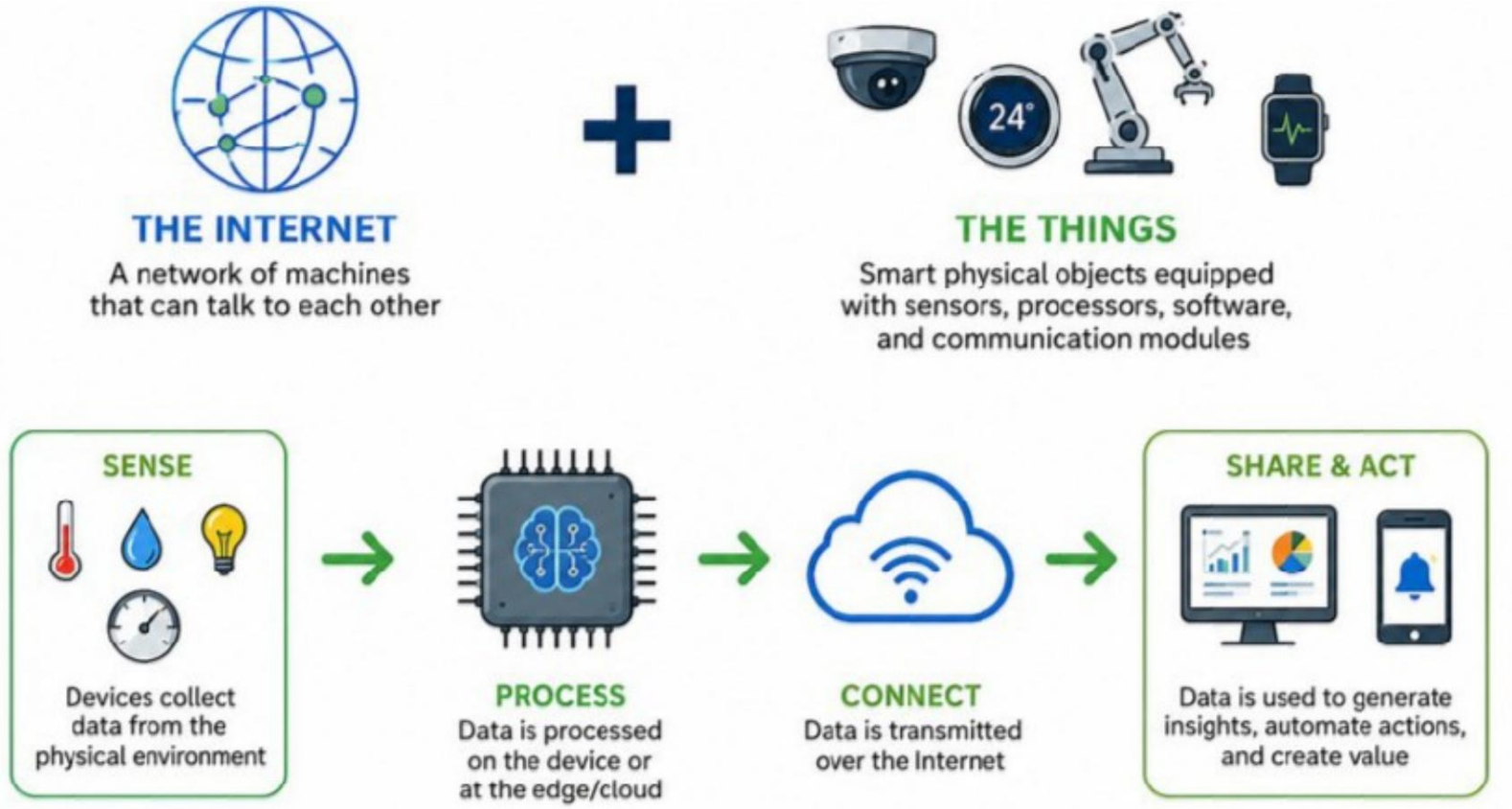
Capteurs, actionneurs et anatomie d'un dispositif connecté

# Le monde connecté




*Villes, santé, transport, industrie, énergie, logistique : tout devient connecté autour du Cloud & de l'IA.*

# L'IoT connecte le monde physique




*IoT = Internet (réseau de machines) + Things (objets physiques équipés de capteurs) — Sense → Process → Connect → Share & Act.*

# Qu'est-ce qu'un dispositif IoT ?

*Un objet physique équipé de capteurs, de logiciel et d'une connectivité réseau pour collecter et échanger des données.*

1

### Capteurs & actionneurs

Collectent les données de l'environnement et exécutent des actions physiques.

2

### Connectivité

Wi-Fi, cellulaire, Bluetooth — nécessite une adresse pour communiquer.

3

### Puissance de traitement

CPU et firmware embarqués pour traiter localement ou transmettre au cloud.

4

### Échange de données

Communication avec d'autres appareils, plateformes ou applications.

# Le dispositif IoT : un pont entre le monde physique et le backend

**Capteur**

Détecte les changements physiques (température, mouvement...)

**Dispositif IoT**

Traite, convertit et achemine la donnée du capteur

**Réseau**

Transporte les données vers le système backend

**Backend (Cloud / Edge)**

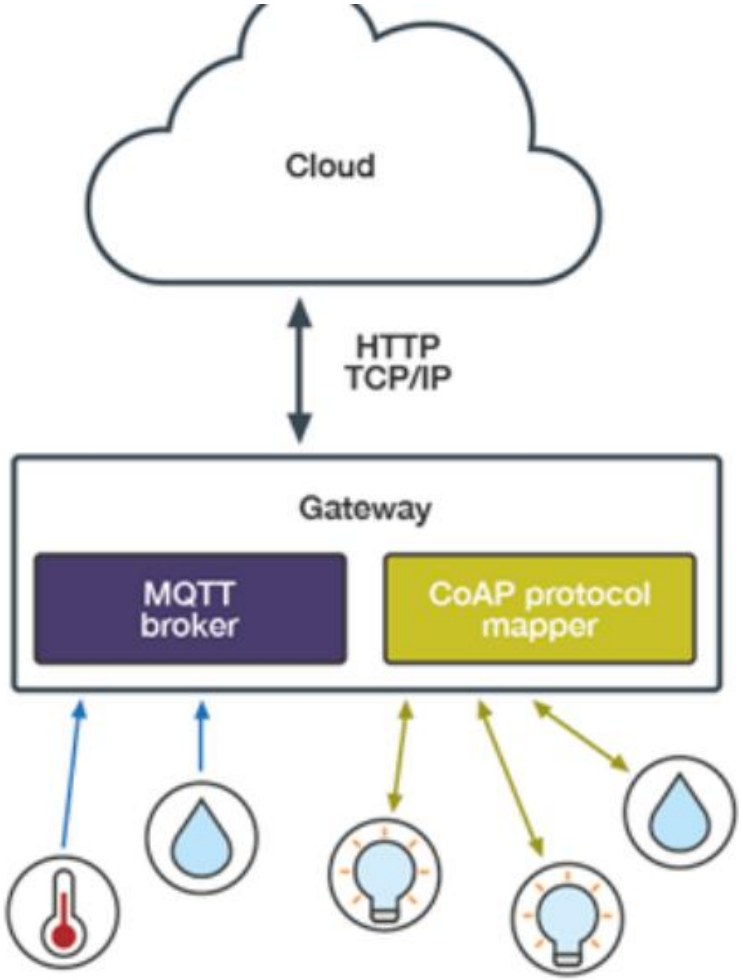
Stocke, analyse et pilote les décisions métier

**Application utilisateur**

Web/mobile pour utilisateurs finaux et administrateurs

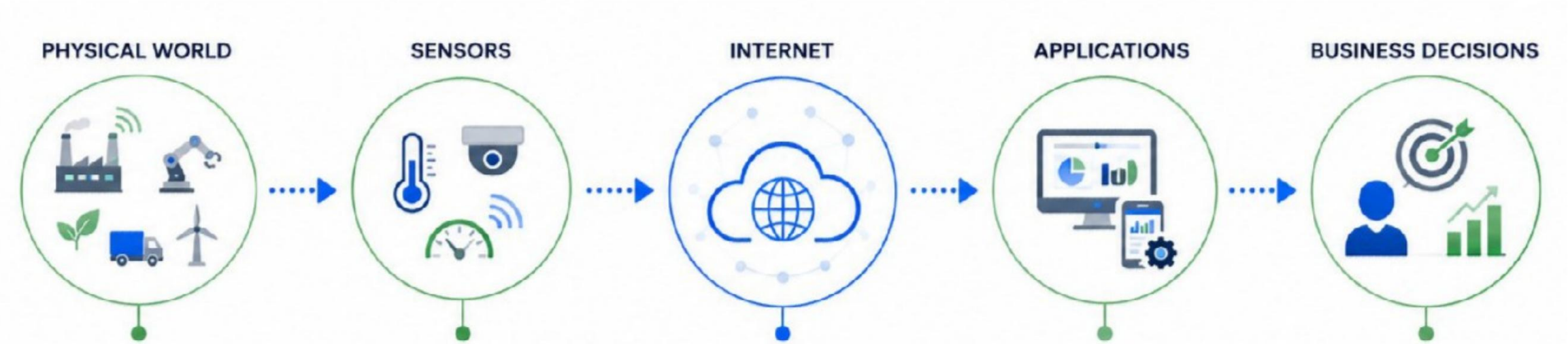
- Le dispositif est le pont qui intègre capteur, actionneur et système backend.
- L'IoT consiste fondamentalement à collecter de l'information
- ... puis à la traiter pour prendre des décisions éclairées.

# L'intégration des capteurs dans l'architecture IoT




*Des capteurs à la passerelle (broker MQTT + CoAP), jusqu'au cloud.*

# Des objets physiques aux décisions business




Objet du monde réel et environnement

Sens et capture Données réelles

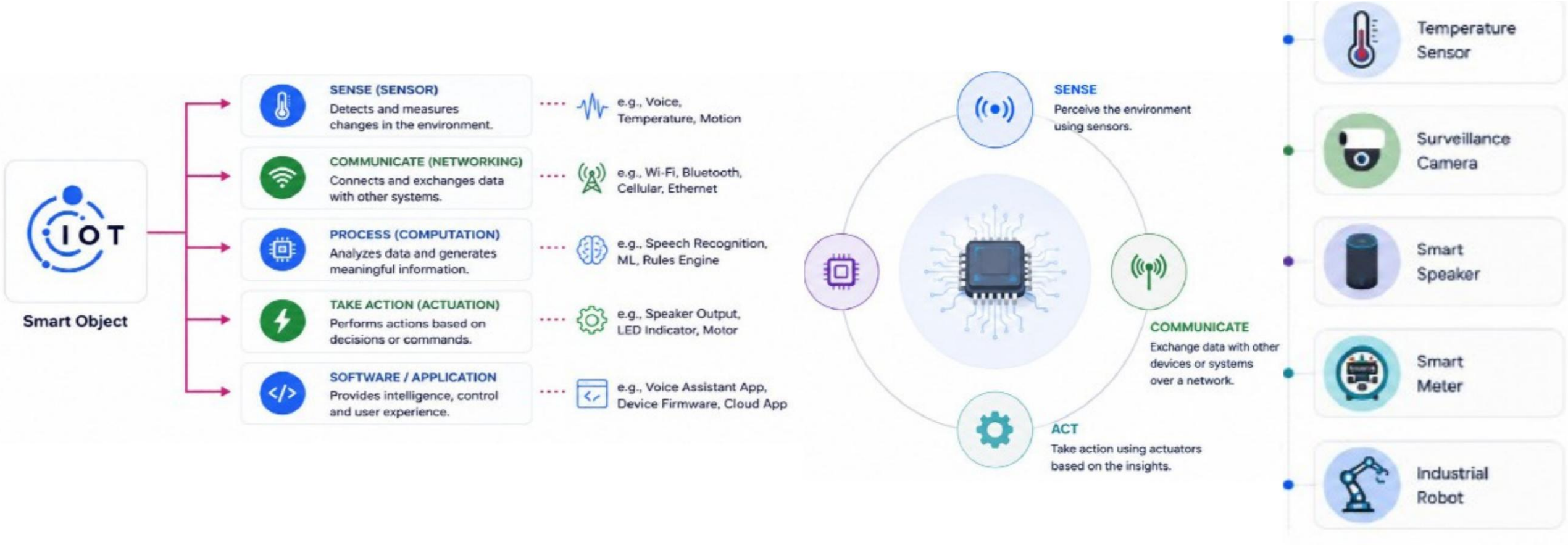
Les données sont transmises sur les réseaux , sécurisés

Traiter, analyser et visualiser

Prendre des décisions Et créer de la valeur

*Monde physique → capteurs → Internet → applications → décisions business : la chaîne de valeur complète de l'IoT.*

# Smart Object : le socle de tout dispositif IoT




**Smart Object soude le lien entre les mondes physique et numérique en détectant, communiquant, traitant, et agissant pour créer un impact concret dans le monde.**

# Exemple : Smart Building




Capteur de température

Envoyez le signal au Centre de contrôle (ROC)

Le ROC envoie des commandements à Arroseur

L'arroseur est ALLUMÉ

*Un capteur de température détecte un incendie → alerte le centre de contrôle → commande l'arroseur → l'arroseur s'active.*

# Capteur vs Actionneur

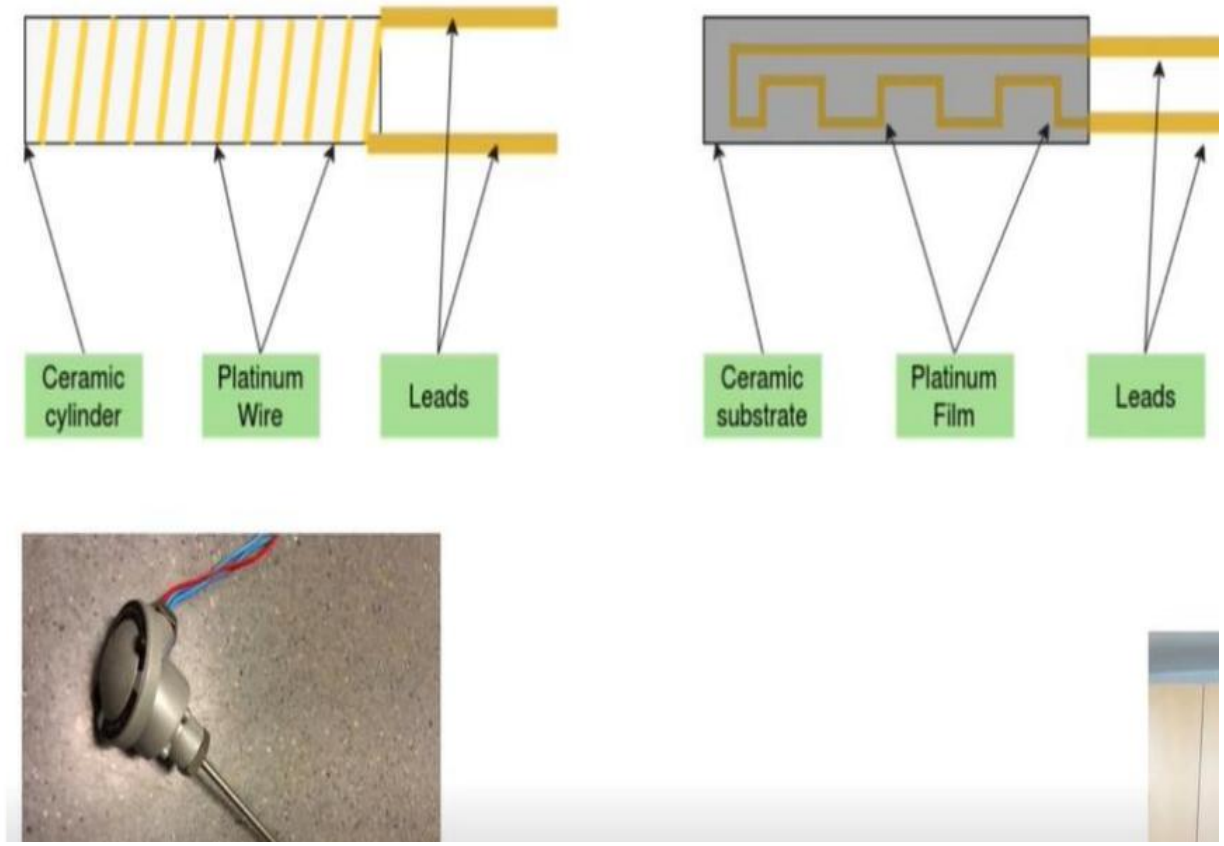
## Capteur — perçoit le monde réel

- Transforme une donnée physique en signal électrique (analogique)
- Sensible aux changements physiques/chimiques → variation de tension, courant, résistance
- Conversion analogique → numérique via échantillonnage et résolution
- Exemple : PT100, sonde de température en platine (100 Ω à 0°C)
- Catégories : environnementaux, entités (GPS, mouvement), densité de particules

## Actionneur — agit sur le monde réel

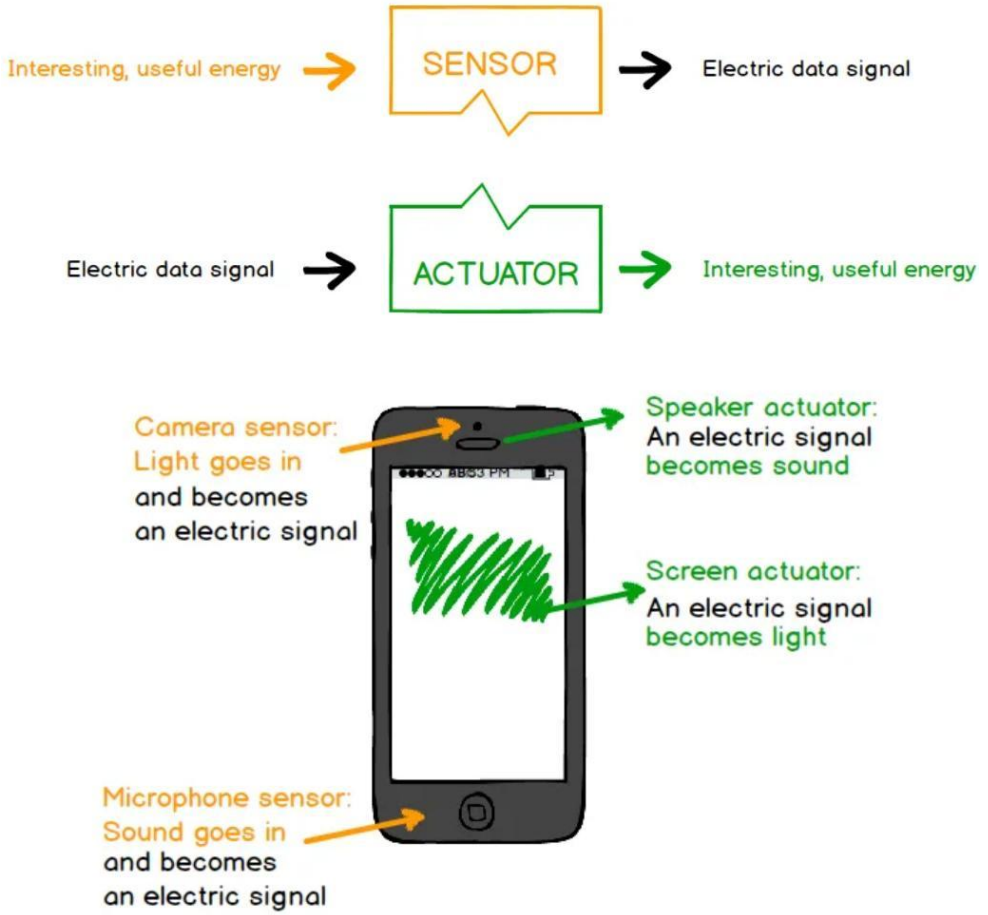
- Convertit un signal électrique en action ou mouvement physique
- Exemple : démarrer une pompe lorsque le niveau d'eau est bas
- Types généraux : hydraulique, pneumatique, mécanique, électrique, thermique
- Déclenché par signal : son, commutation (relais), mouvement (servo, solénoïde)
- Exemples courants : haut-parleurs, écrans, moteurs, vannes

# Le capteur PT100 en détail




*Construction du PT100 : fil de platine (à gauche) ou film de platine (à droite), et exemple réel de sonde.*

# Capteur et actionneur : l'exemple du smartphone



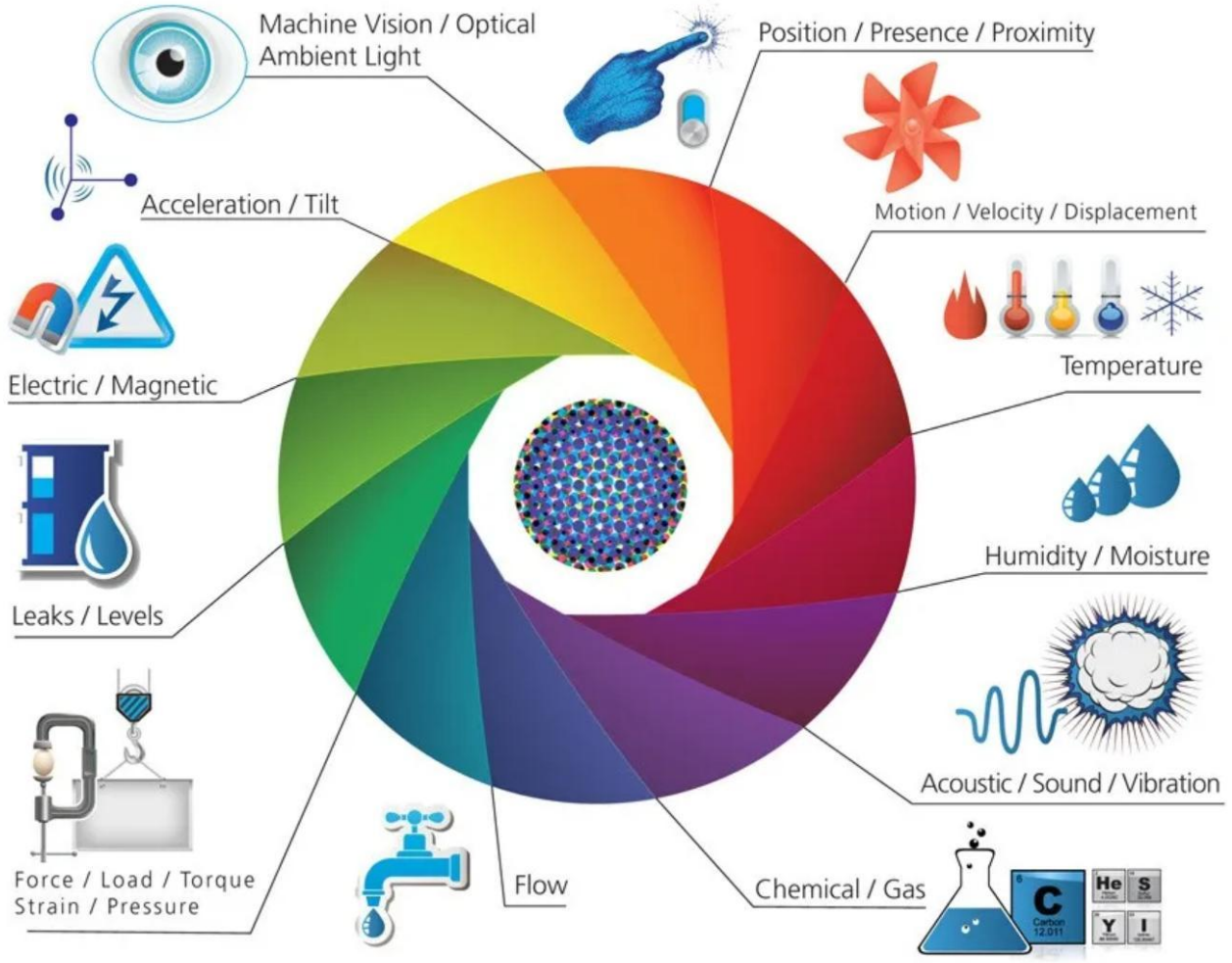

*Caméra et microphone convertissent le monde réel en signal électrique ; écran et haut-parleur font l'inverse.*

# Catégories de capteurs IoT

| Catégorie | Exemples de grandeurs mesurées | Types de capteurs |
|---|---|---|
| Environnementaux | Température, humidité, pression | Thermistance, thermocouple, hygromètre |
| Entités | Localisation, mouvement | GPS, accéléromètre, gyroscope, tachymètre |
| Densité de particules | Qualité de l'air, fumée, gaz | Infrarouge, électrochimique, photoélectrique |

*Plus de 10 milliards de dispositifs IoT dans le monde aujourd'hui — 30 milliards attendus d'ici 2030.*

# Panorama des types de capteurs




*De la position au chimique : la grande diversité des familles de capteurs IoT.*

# Tout devient intelligent



*Villes, transports, santé, énergie : les capteurs se déploient partout, portés par la **5G/6G** et la miniaturisation.*

# Types de dispositifs IoT : usage général vs sur-mesure

| Catégorie | Caractéristiques | Exemples |
|---|---|---|
| Usage général (apprentissage / test) | IDE, SDK, pilotes fournis ; connecte capteurs & actionneurs facilement | ESP32, Raspberry Pi 4, Arduino Uno |
| Sur-mesure (production) | Capteurs embarqués dédiés, sécurité renforcée, IP68, tamper-proofing | InnoSent, DSEGenset, Google Nest, Alexa |

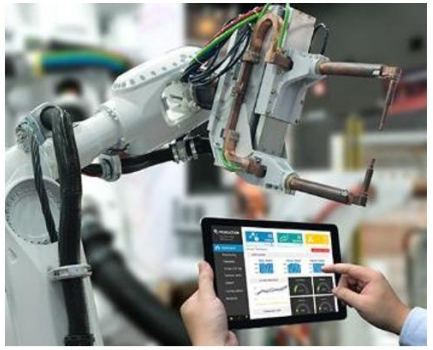
*Processeurs typiques : 8 bits (2-40 MHz) → 32 bits multicœur. Logiciel : assembleur/C sur petits devices, FreeRTOS / MbedOS / RTLinux sur devices plus puissants.*

# Les grandes familles de dispositifs IoT en images


Maison intelligente



Wearables

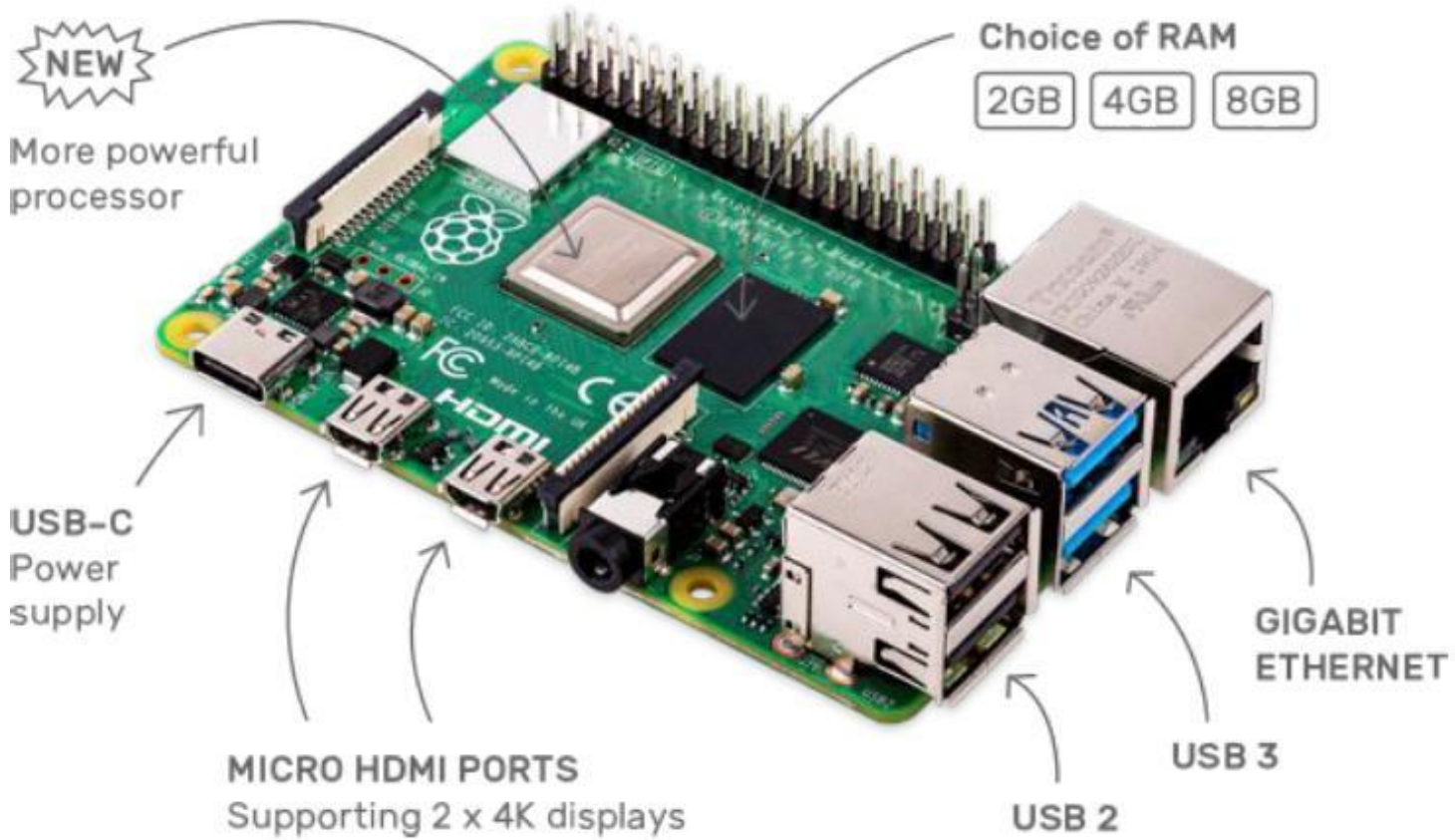

Industriel (IIoT)


Automobile


Agriculture

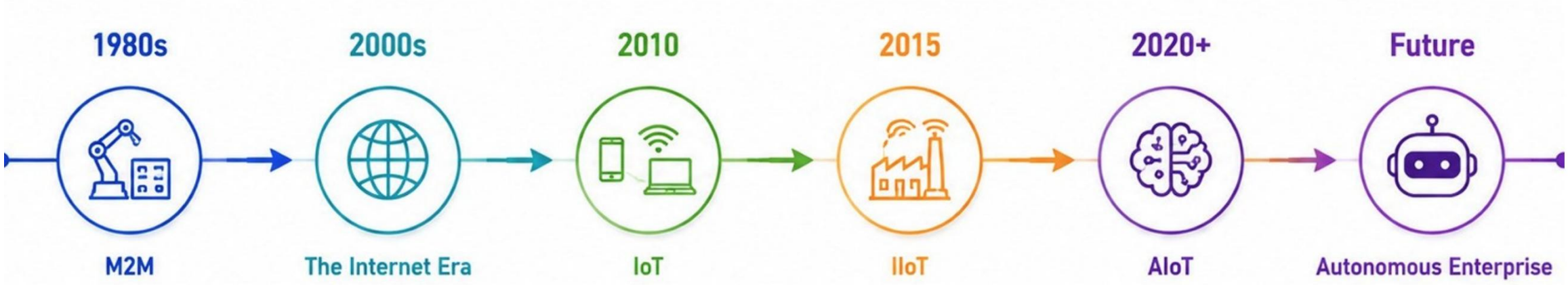
*Maison, santé/wearables, industrie, automobile, agriculture — un même principe, des usages multiples.*

# Exemple de dispositif à usage général : Raspberry Pi 4



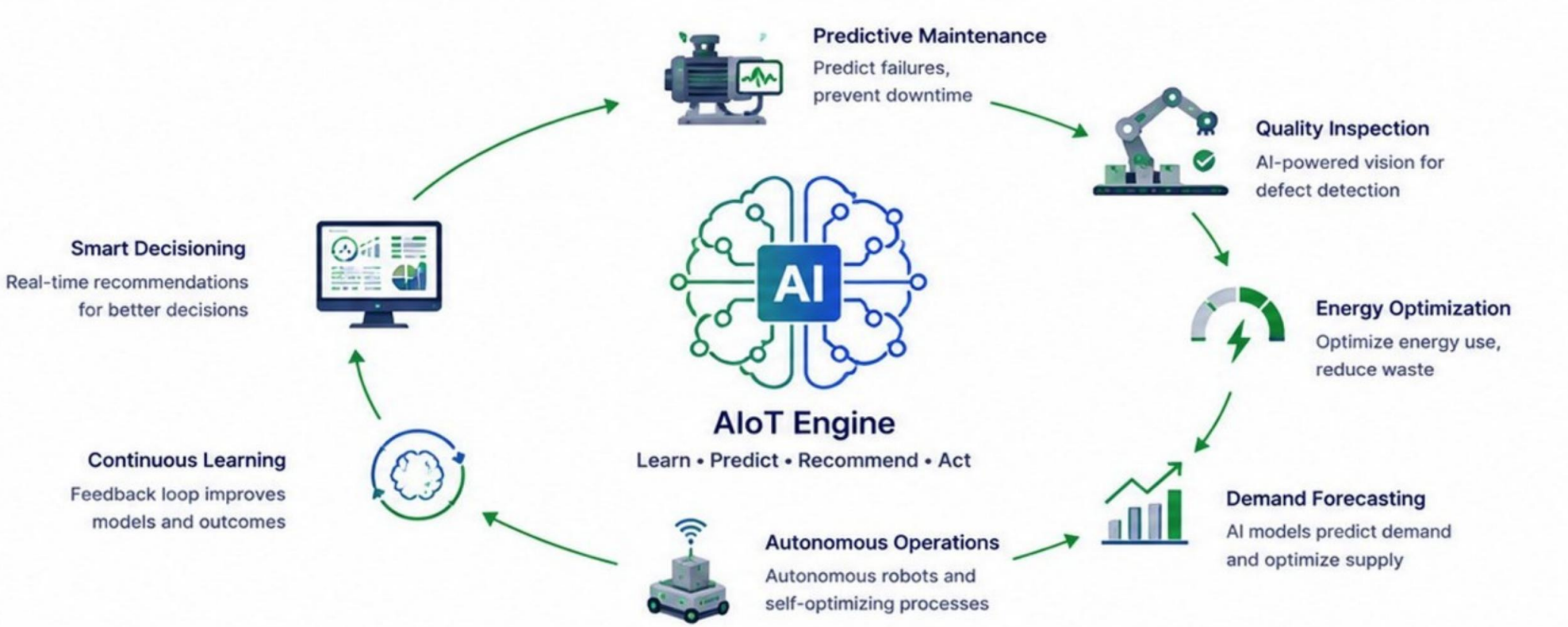

*RAM configurable, USB-C, HDMI, Ethernet gigabit — une plateforme complète pour prototyper un dispositif IoT.*

# L'évolution de l'IoT : du M2M à l'entreprise autonome



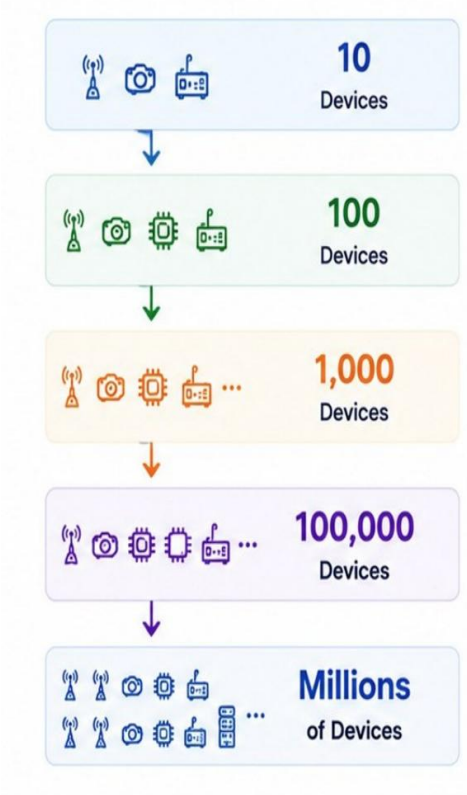

*M2M (1980s) → Internet → IoT (2010) → IIoT (2015) → AIoT (2020+) → Entreprise autonome : le terme AIoT que nous utilisons tout au long de ce cours.*

# AIoT : ramener l'intelligence aux réseaux

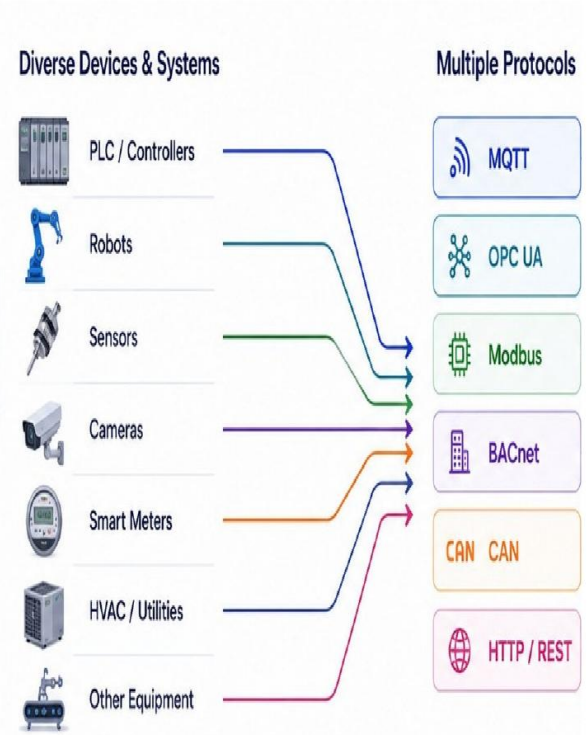



*Learn → Predict → Recommend → Act : maintenance prédictive, inspection qualité, optimisation énergétique, prévision de la demande, opérations autonomes.*

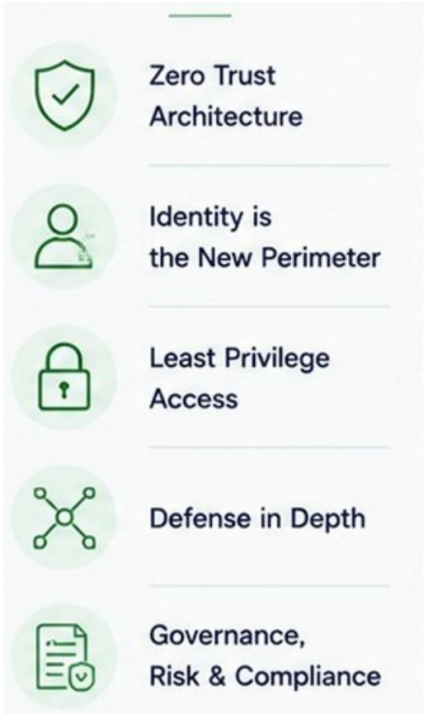
# Les 4 grands défis de conception d'une solution IoT




Conception at Scale




Interopérabilité




Sécurité




Latence et Perf

*Conception à l'échelle, interopérabilité, sécurité, latence & performance — les 4 défis que ce cours va résoudre un à un, à partir du Module 2.*

# IoT en Application:  ramener l'intelligence aux réseaux



MODULE 2

# Réseaux IoT

Protocoles, topologies et technologies de connectivité

# Réseau Client-Serveur vs Réseau IoT

## Client – Serveur (traditionnel)

- Connexion stable et fiable supposée
- Architecture simple : requête → réponse
- Protocoles : HTTP/HTTPS, REST, WebSockets
- Nombre de clients limité, trafic prévisible
- Dispositifs toujours alimentés (PC, serveurs)

## IoT

- Connectivité imprévisible, intermittente, faible bande passante
- Architecture multi-couches : Device → Gateway → Edge → Cloud
- Protocoles multiples : BLE, Zigbee, LoRaWAN, NB-IoT, MQTT, CoAP…
- Échelle massive : milliers à millions de dispositifs
- Dépendance batterie : l'efficacité énergétique est critique

*L'IoT n'est pas qu'une question de connectivité : c'est gérer la diversité, la fiabilité et la scalabilité à très grande échelle.*

# Les 3 modes de communication IoT

1. **Device-to-Device (D2D)**
   Communication directe sans intermédiaire. Ex : un capteur de température signale instantanément un ventilateur.

2. **Device-to-Gateway (D2G)**
   Les dispositifs se connectent à une passerelle pour l'agrégation et le traitement local. Ex : capteurs industriels vers un contrôleur edge.

3. **Device-to-Cloud (D2C)**
   Communication directe avec le cloud pour l'analytique avancée. Ex : compteurs intelligents envoyant leurs données pour facturation.

# Les 3 éléments clés du réseau IoT

1

**Interfaces réseau**

Composants matériels/logiciels connectant le dispositif au réseau — filaires (USB, Ethernet, RS232/485) ou sans fil (BLE, Zigbee, Wi-Fi, LoRaWAN, 5G, NB-IoT).

2

**Topologies réseau**

Point-à-point (2 devices directs), maillage (Mesh — chemins multiples), LAN sans fil (étoile autour d'un point d'accès).

3

**Protocoles de communication**

Règles régissant la transmission : traditionnels (HTTP, FTP), messagerie (SMTP, XMPP), spécialisés IoT (MQTT, CoAP).

# Le paysage des protocoles par couche OSI

| Couche OSI | Rôle | Protocoles / Technologies |
|---|---|---|
| Physique | Transmission des bits | Filaire : Ethernet, RS232, USB — Sans fil : BLE, Zigbee, LoRaWAN, 5G, NB-IoT |
| Liaison de données | Adressage MAC | Wi-Fi MAC, Zigbee MAC, Bluetooth MAC, Ethernet MAC |
| Réseau | Adressage et routage | IPv4, IPv6 |
| Transport | Fiabilité ou rapidité | TCP (fiable), UDP (rapide), RTSP (streaming temps réel) |
| Application | Échange applicatif | HTTP/HTTPS, MQTT (pub/sub), CoAP (contraint), Modbus, FTP, SMTP |

# Comparatif des technologies sans fil

| Technologie | Portée | Débit | Consommation | Coût |
|---|---|---|---|---|
| Bluetooth LE | 1–100 m | ≤ 2 Mbps | Très faible | Faible |
| Zigbee (802.15.4) | 10–100 m (mesh) | 20–250 kbps | Faible | Faible |
| Wi-Fi (802.11) | ~25 m indoor | 7 Mbps–9.6 Gbps | Modérée | Modérée à élevée |
| LoRaWAN | 5–10 km | 0.3–50 kbps | Ultra-faible | Faible |
| NB-IoT | 1–10 km | 20–200 kbps | Très faible (10 ans batterie) | Faible |
| **LTE (4G), RedCap (5G)** | Zone étendue | ≤ 100 Mbps | Élevée | Élevé |

*Tendance : passage d'un protocole unique à une connectivité hybride spécialisée par cas d'usage (Wi-Fi basse consommation, réseaux maillés, etc.).*

# 4 facteurs clés dans le choix d'un protocole

1

**Coût**

Coût de traitement de la pile réseau et du transceiver selon la distance.

2

**Bande passante**

Optimale selon la topologie, la distance et les pics de charge.

3

**Consommation énergétique**

Traitement et transfert de données, filaire ou sans fil.

4

**Attributs d'infrastructure**

Sécurité, latence, connexions simultanées supportées.

MODULE 3

# MQTT — le protocole standard de l'IoT

Publish/Subscribe, Quality of Service et sécurité

# Pourquoi Publish/Subscribe plutôt que la communication directe ?

## Communication directe (point à point)

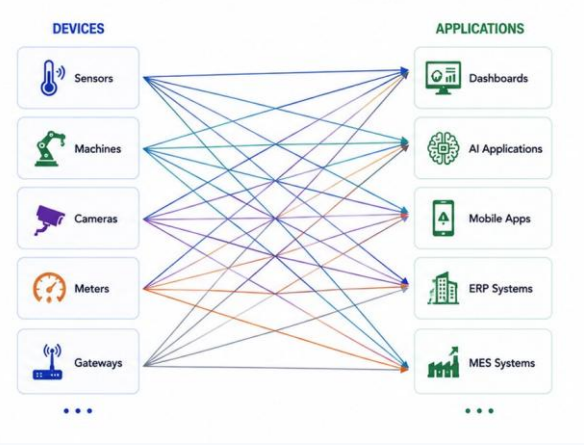
- Chaque dispositif parle directement à chaque application
- Explosion du nombre de connexions (5 000 capteurs × 20 apps…)
- Couplage fort entre dispositifs et applications
- Complexe à déployer, maintenir et faire évoluer
- Latence et surcharge réseau élevées

## Publish / Subscribe via un Broker

- Toute la communication transite par un broker central
- Les éditeurs (publishers) ne connaissent pas les abonnés
- Ajout de nouveaux devices/apps sans impact sur les autres
- Scalable à des millions de dispositifs
- Livraison fiable grâce à la Qualité de Service (QoS)

*Une approche découplée qui scale : c'est la base de l'architecture AIoT d'entreprise.*

# Les 3 rôles du modèle MQTT




1

**1. Publisher**

Dispositifs qui publient des données : capteurs, machines/PLC, caméras, compteurs.

2

**2. Broker**

Serveur central : reçoit les messages, filtre par topic, gère les abonnements et le QoS.

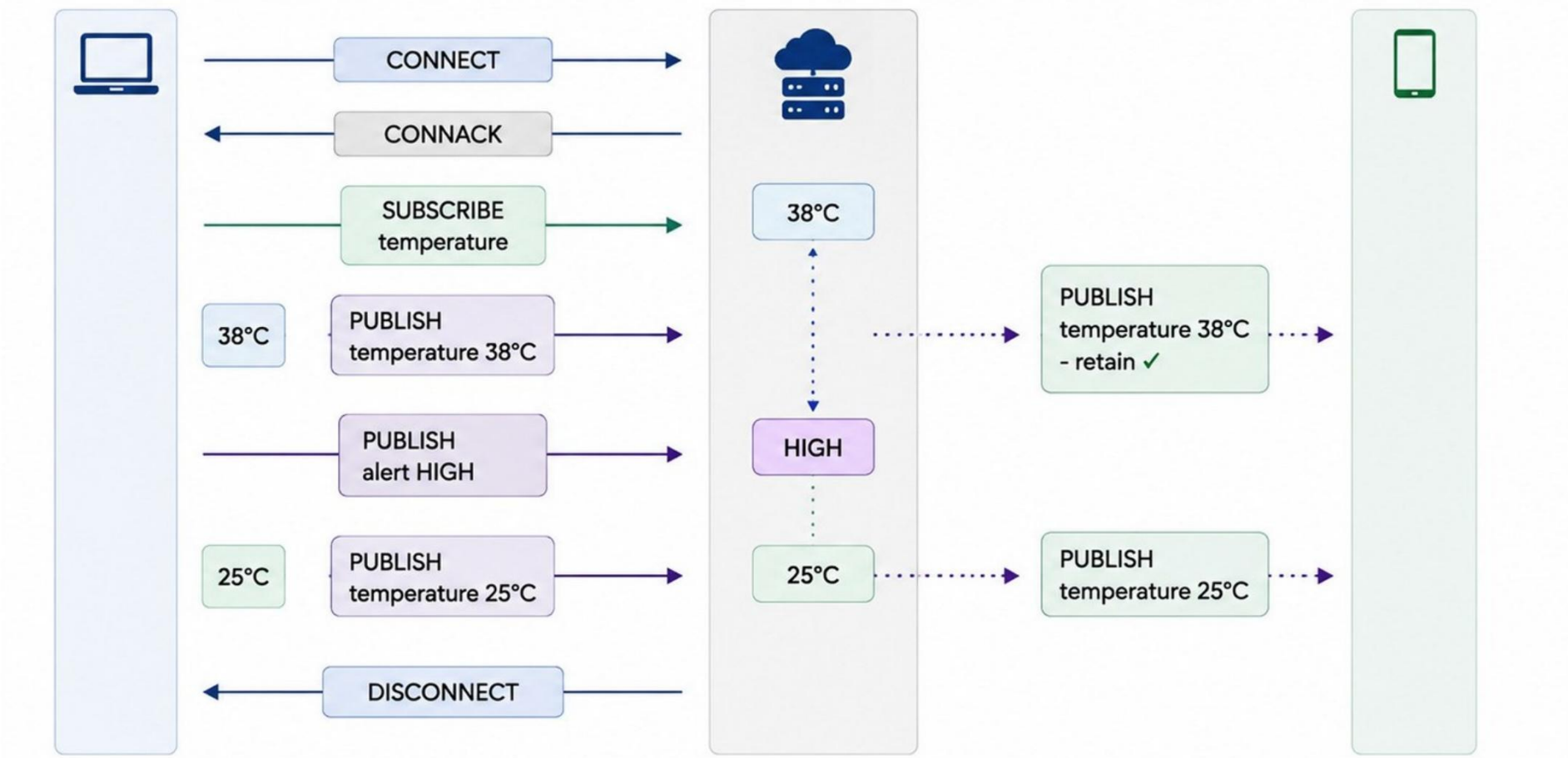
3

**3. Subscriber**

Applications abonnées : dashboards, moteurs IA, apps mobiles, systèmes ERP/MES.

*Les publishers ne savent pas qui les écoute ; les subscribers ne savent pas qui publie. Le broker gère tout entre les deux.*

# Le mécanisme du protocole MQTT, message par message




*CONNECT/CONNACK établissent la session ; SUBSCRIBE enregistre l'intérêt ; chaque PUBLISH est routé par le broker vers les abonnés du topic.*

# Les Topics : la boîte aux lettres intelligente

1. **Structure hiérarchique**

   factory1/production/machine12/temperature — organisée en niveaux séparés par "/".

2. **Wildcard niveau unique ( + )**

   factory1/+/machine12/temperature → correspond à un seul niveau, quel que soit son nom.

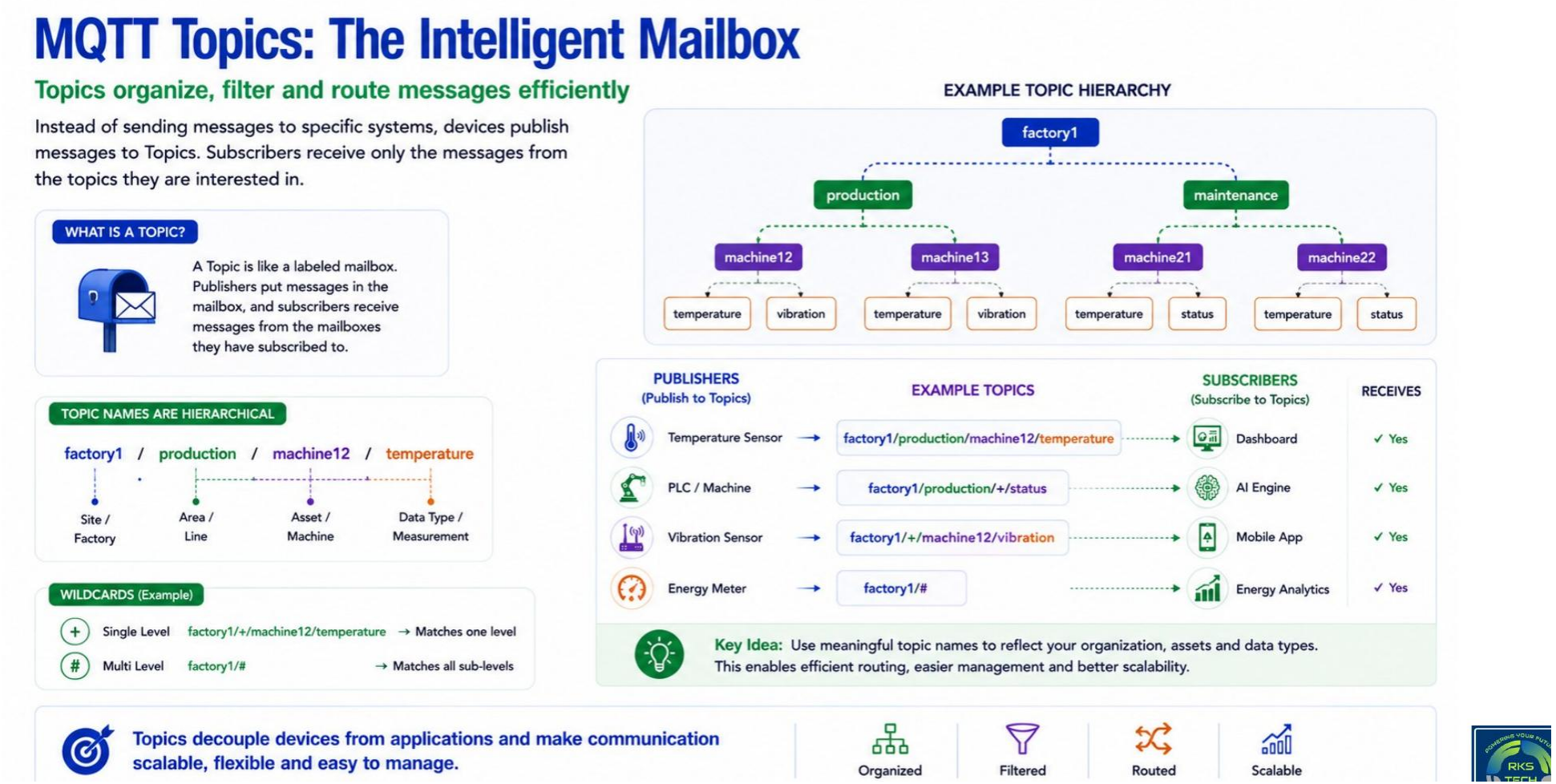
3. **Wildcard multi-niveaux ( # )**

   factory1/# → correspond à tous les sous-niveaux ; à utiliser avec prudence en production.

4. **Topics système ( $SYS/ )**

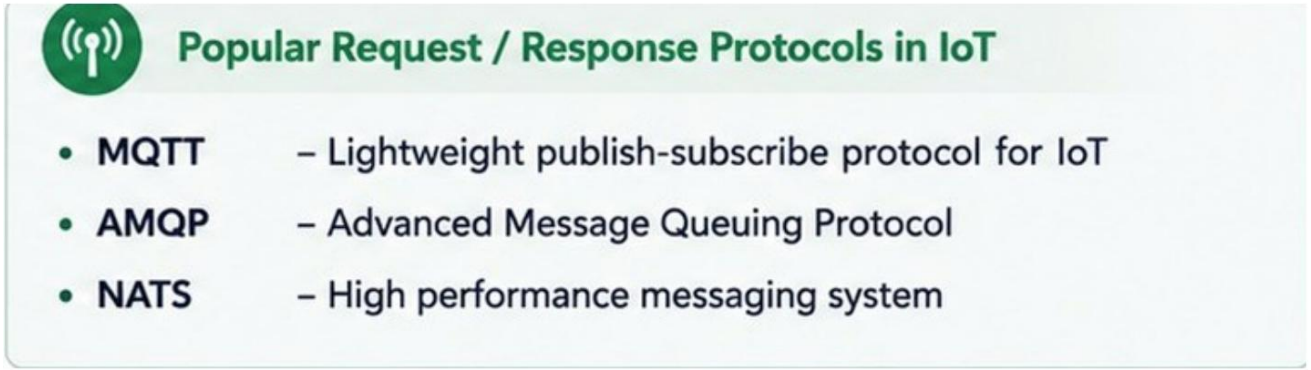
   Réservés aux statistiques internes du broker (clients connectés, messages envoyés, etc.).

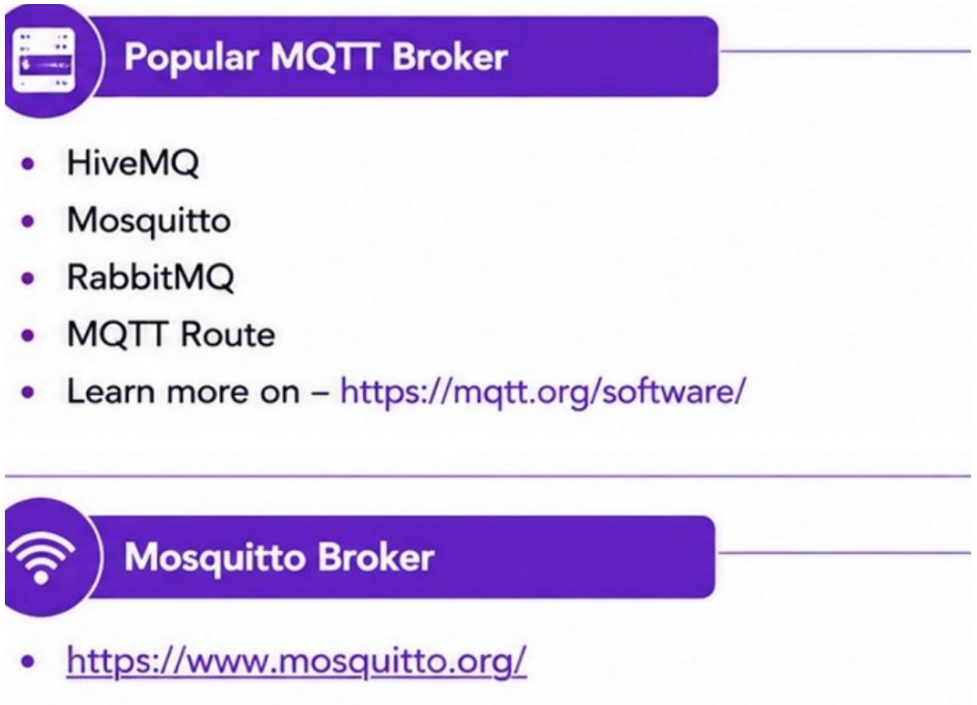
# Les topics MQTT : la boîte aux lettres intelligente




*Hiérarchie de topics, wildcards (+ niveau unique, # multi-niveaux) et routage publisher → subscriber.*

# Brokers MQTT populaires et protocoles comparés




*MQTT vs AMQP vs NATS — et les brokers de référence : Mosquitto, HiveMQ, RabbitMQ, MQTT Route.*

# QoS : les 3 niveaux de garantie de livraison

| Niveau | Garantie | Mécanisme | Cas d'usage typique |
|---|---|---|---|
| QoS 0 | Au plus une fois (best effort) | Aucun accusé de réception | Télémétrie, données de capteurs périodiques |
| QoS 1 | Au moins une fois | Accusé PUBACK (doublons possibles) | Alertes, notifications, suivi de stock |
| QoS 2 | Exactement une fois | Poignée de main en 4 étapes | Commandes critiques, transactions, arrêt d'urgence |

***Règle d'or** : choisir le QoS le plus bas qui satisfait le besoin métier — cela réduit charge réseau, latence et coût.*

# Comparatif détaillé des 3 niveaux de QoS

## Comparing MQTT QoS Levels

### Three Levels of Reliability for Every Business Need

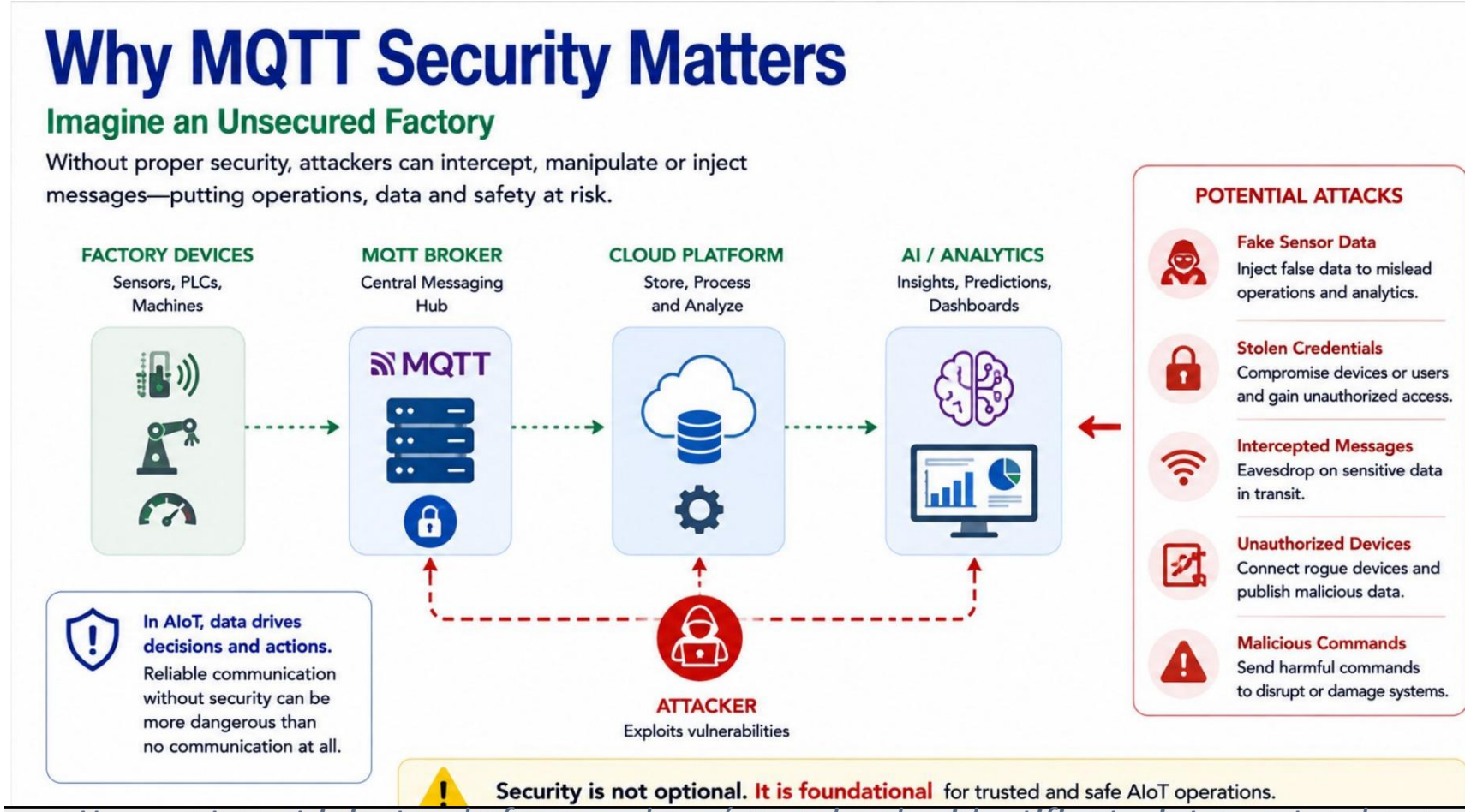
| Feature | QoS 0<br>At Most Once | QoS 1<br>At Least Once | QoS 2<br>Exactly Once |
|---|---|---|---|
| Delivery Guarantee | Best Effort<br>Message may be lost | At Least Once<br>Message will be delivered at least once | Exactly Once<br>Message will be delivered exactly once |
| Duplicate Messages | No | Possible | No |
| Acknowledgment | None | PUBACK | Four-step Handshake<br>(PUBREC, PUBREL, PUBCOMP) |
| Speed / Latency | Fastest | Moderate | Slowest |
| Network Overhead | ● Lowest | ●● Medium | ●●● Highest |
| Reliability Level | ★ Low | ★★ Medium | ★★★ High |
| Best For | Telemetry, sensor data, periodic updates | Alerts, notifications, status updates, tracking | Critical commands, financial transactions, shutdown signals |

**Key Takeaway:** Higher QoS levels provide stronger delivery guarantees but come with increased acknowledgments, latency and network overhead. Choose the lowest QoS that meets your business requirement.

| Speed | Reliability | Overhead |
|---|---|---|
| QoS 0 > QoS 1 > QoS 2 | QoS 0 < QoS 1 < QoS 2 | QoS 0 < QoS 1 < QoS 2 |

*Vitesse, fiabilité et surcharge réseau évoluent en sens opposés selon le niveau de QoS choisi.*

# Pourquoi la sécurité MQTT est critique : le scénario de l'usine non sécurisée




*Sans sécurité, un attaquant peut injecter de fausses données, voler des identifiants, intercepter des messages ou envoyer des commandes malveillantes.*

# Sécuriser MQTT : 3 piliers

1

**1. Authentification — Qui êtes-vous ?**

Vérifie l'identité des dispositifs, utilisateurs et applications avant connexion au broker (certificats X.509, OAuth, identifiants forts).

2

**2. Chiffrement — Le message est-il lisible ?**

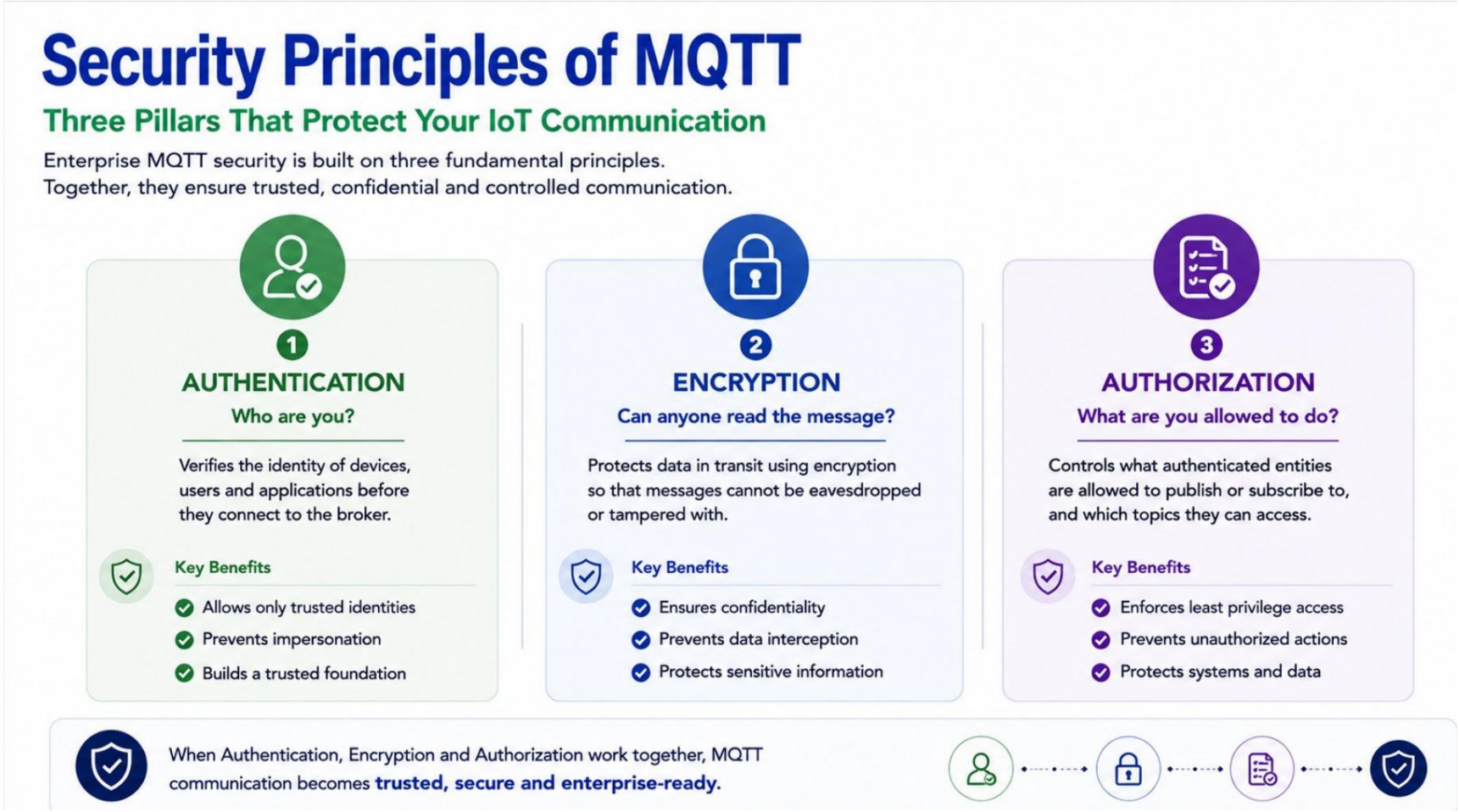
Protège les données en transit via TLS (1.2 ou supérieur) pour empêcher interception et falsification.

3

**3. Autorisation — Que pouvez-vous faire ?**

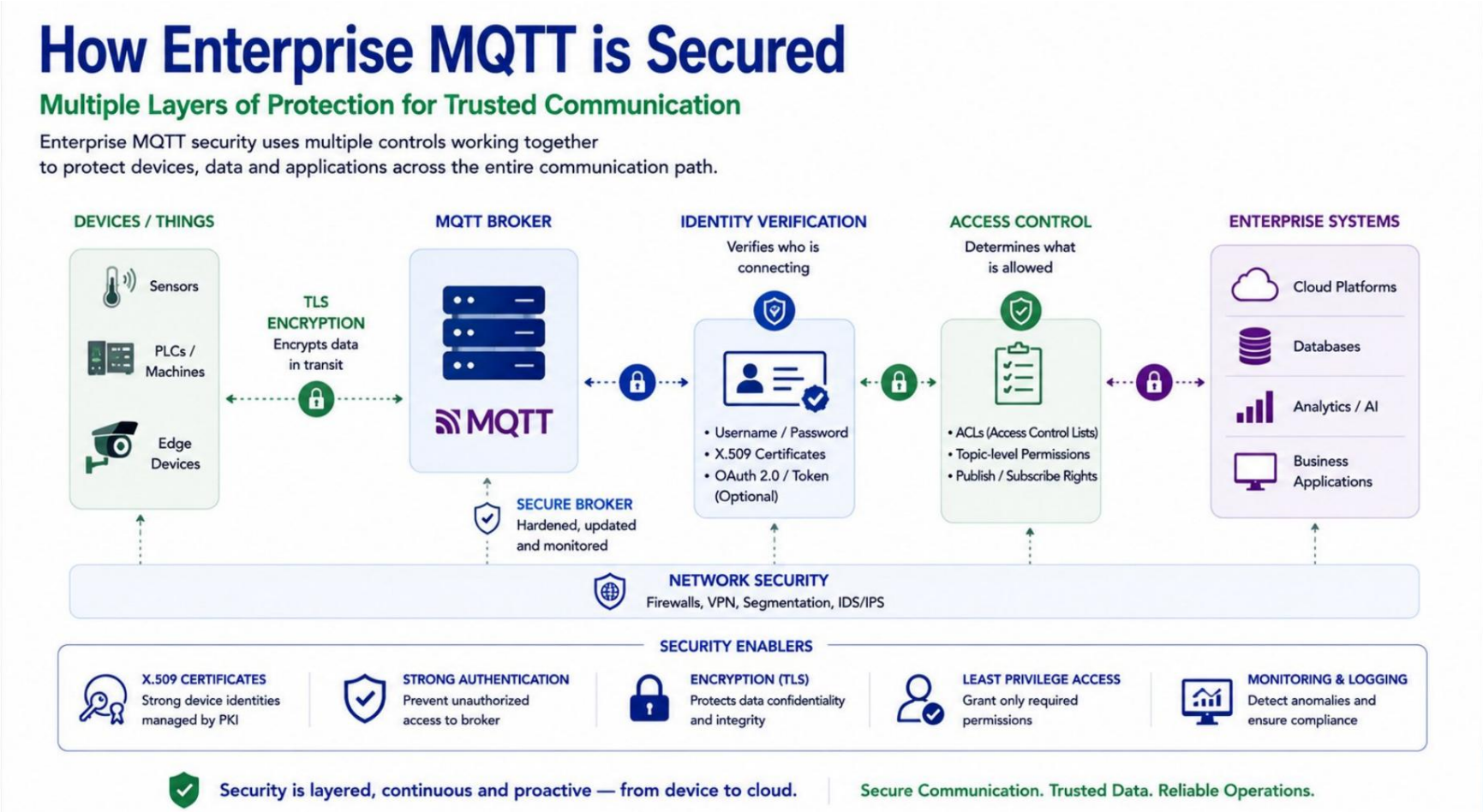
Contrôle ce que chaque entité authentifiée peut publier/souscrire via des ACL au niveau des topics (principe du moindre privilège).

# Les 3 piliers de la sécurité MQTT en détail




*Authentification, chiffrement, autorisation — ensemble, ils rendent la communication MQTT fiable, sécurisée et prête pour l'entreprise.*

# Comment MQTT est sécurisé en entreprise, de bout en bout




*Chiffrement TLS, vérification d'identité, contrôle d'accès et sécurité réseau protègent chaque étape, du dispositif au système d'entreprise.*

# 8 bonnes pratiques de sécurité MQTT en entreprise

1. **Chiffrement TLS**
   Toujours activer TLS 1.2+ pour protéger le trafic MQTT.

2. **Authentification forte**
   Certificats, OAuth ou identifiants robustes obligatoires.

3. **Certificats de dispositif**
   X.509 pour une identité unique et évolutive.

4. **Moindre privilège**
   N'accorder que les permissions strictement nécessaires.

5. **Broker sécurisé**
   Mise à jour régulière, pare-feu, désactivation de l'accès anonyme.

6. **Rotation des identifiants**
   Renouveler régulièrement mots de passe, tokens, certificats.

7. **Journalisation & supervision**
   Surveiller connexions et activités pour détecter les anomalies.

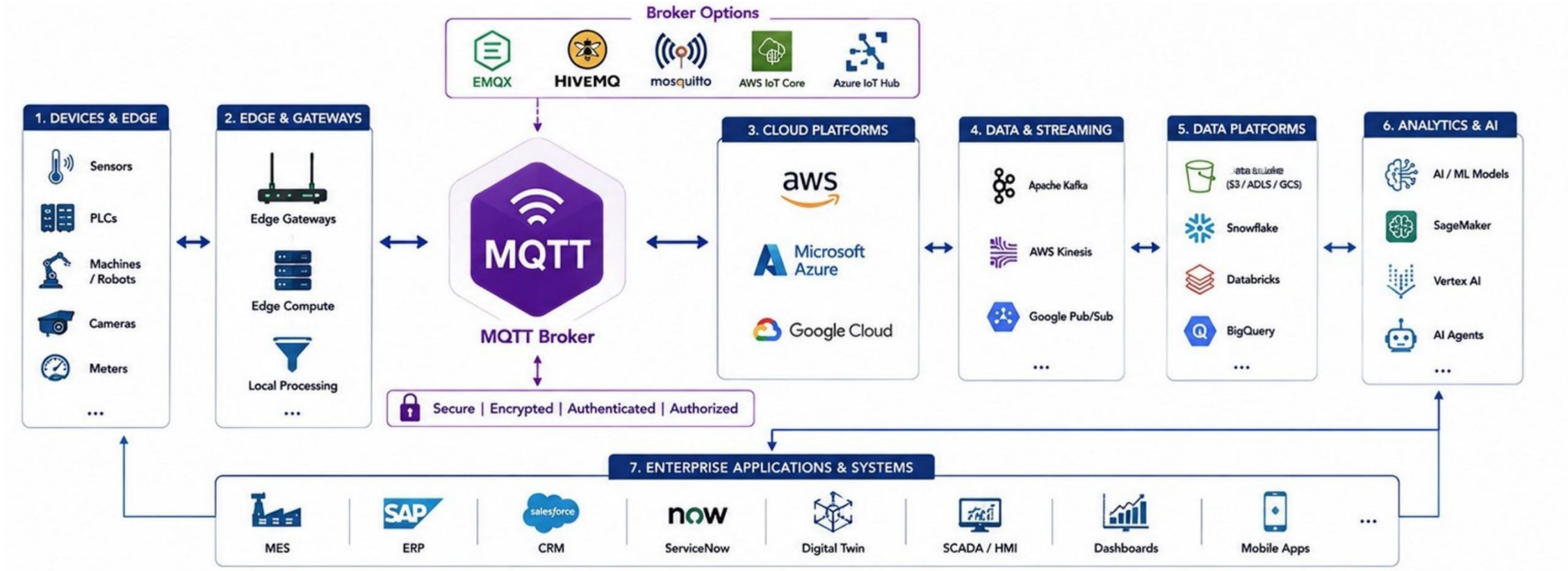
8. **Blocage de l'accès anonyme**
   Chaque client doit être authentifié et autorisé.

# Synthèse visuelle : les 8 bonnes pratiques de sécurité MQTT




*La sécurité se construit dès l'architecture — pas après le déploiement.*

# L'intégration de MQTT dans l'écosystème d'entreprise




*Des devices/edge au broker MQTT, puis vers le cloud, le streaming, les plateformes de données, l'analytique/IA et les applications d'entreprise.*

# L'IoT dans l'écosystème d'entreprise: manipulation d'une  grue à distance

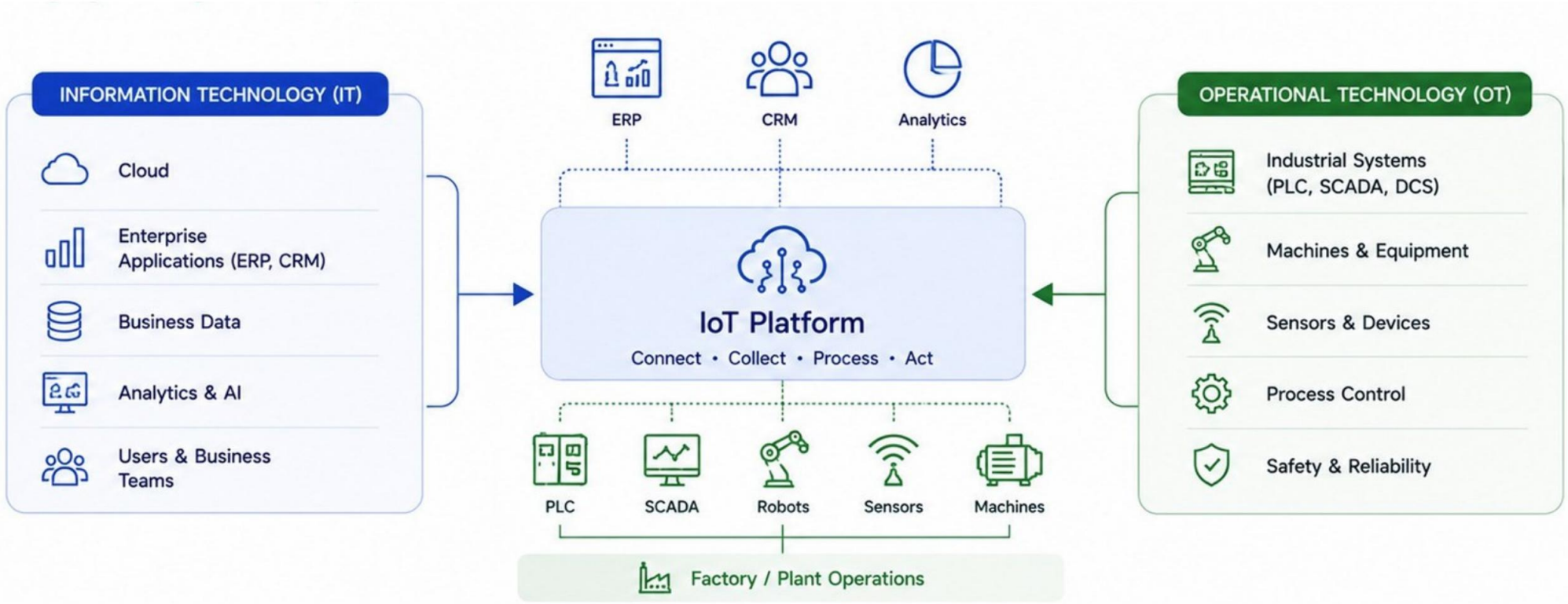


MODULE 4

# Cloud & Edge Computing

Architecture multi-couches pour le traitement des données IoT

# Pourquoi a-t-on besoin d'une plateforme IoT ?




*La plateforme IoT connecte, collecte, traite et agit — au croisement de l'IT (cloud, ERP, analytique) et de l'OT (PLC, SCADA, machines, capteurs).*

# L'architecture IoT multi-couches

**Cloud Computing**
Centre de données central — analytique globale, stockage long terme

**Fog Computing**
Centre de données régional — traitement intermédiaire

**Edge Computing**
Traitement à proximité de l'utilisateur — faible latence

**Mist Computing**
Extrême périphérie — traitement au plus proche du capteur

**Device Computing**
Traitement directement sur le dispositif

- Le traitement des données a lieu à plusieurs niveaux, du device au cloud.
- Chaque couche répond à un compromis latence / puissance / coût différent.

# Cloud Computing dans l'IoT

## Rôle du Cloud

- Plateforme centralisée, toujours disponible, hautement scalable
- Ingestion de données, stockage long terme
- Entraînement de modèles IA/ML
- Gestion des dispositifs et intégration aux systèmes d'entreprise
- Tableaux de bord et analytique avancée

## Limites d'une architecture 100% Cloud

- Latence élevée pour un contrôle en temps réel
- Dépendance totale à la connectivité internet
- Coûts de transmission de données importants
- Enjeux de confidentialité et de résidence des données
- Inadapté aux systèmes de sécurité critiques

*Ces limites conduisent naturellement vers l'Edge Computing.*

# Edge Computing : la vitesse, l'autonomie, la résilience

1. **Filtrage & agrégation**

   Réduction du volume de données avant transmission.

2. **Décisions par seuil**

   Exécution de règles locales en temps réel.

3. **Traduction de protocole**

   Interopérabilité entre dispositifs hétérogènes.

4. **Stockage temporaire**

   Continuité de service en cas de coupure réseau.

5. **Opérations fail-safe**

   Sécurité maintenue même hors ligne.

6. **Inférence IA à l'edge**

   Prédictions locales sans dépendre du cloud.

*Traiter les données au plus près de leur source, sur le dispositif lui-même ou une passerelle proche.*

# Device vs Edge vs Cloud : quand utiliser quoi ?

| Couche | Rôle principal | Utiliser quand… |
|---|---|---|
| Device | Détecter & agir | Action immédiate requise sur le capteur/actionneur lui-même |
| Edge | Intelligence locale | Faible latence requise, connectivité peu fiable, décisions de sécurité critique |
| Cloud | Intelligence globale | Analytique long terme, corrélation multi-sites, entraînement IA/ML, reporting métier |

*Le Cloud et l'Edge sont complémentaires, pas concurrents : ensemble ils apportent vitesse + échelle + intelligence.*

APPLICATIONS

# Cartographie des cas d'usage réels

- Usine intelligente — Edge : surveillance machine et détection d'anomalies · Cloud : maintenance prédictive
- Ville intelligente — Edge : contrôle des feux de circulation · Cloud : optimisation et planification du trafic
- Santé connectée — Edge : alertes patient en temps réel · Cloud : historique patient et analytique

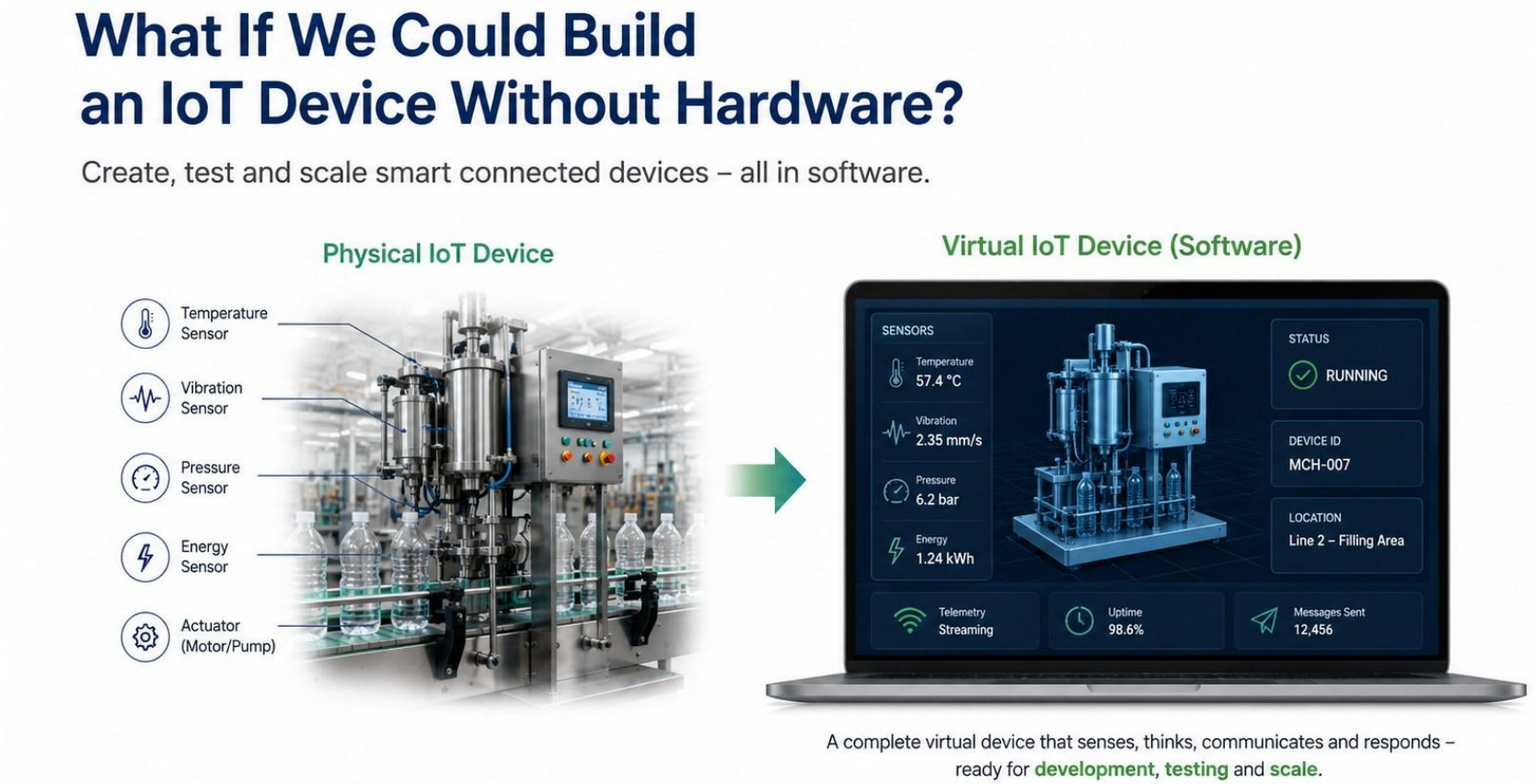
**5 erreurs fréquentes à éviter : tout envoyer au cloud, ignorer l'intelligence edge, sur-complexifier la logique edge, sécurité faible à l'edge, absence de stratégie de repli en cas de panne.**

MODULE 5

# Construire un dispositif AIoT virtuel

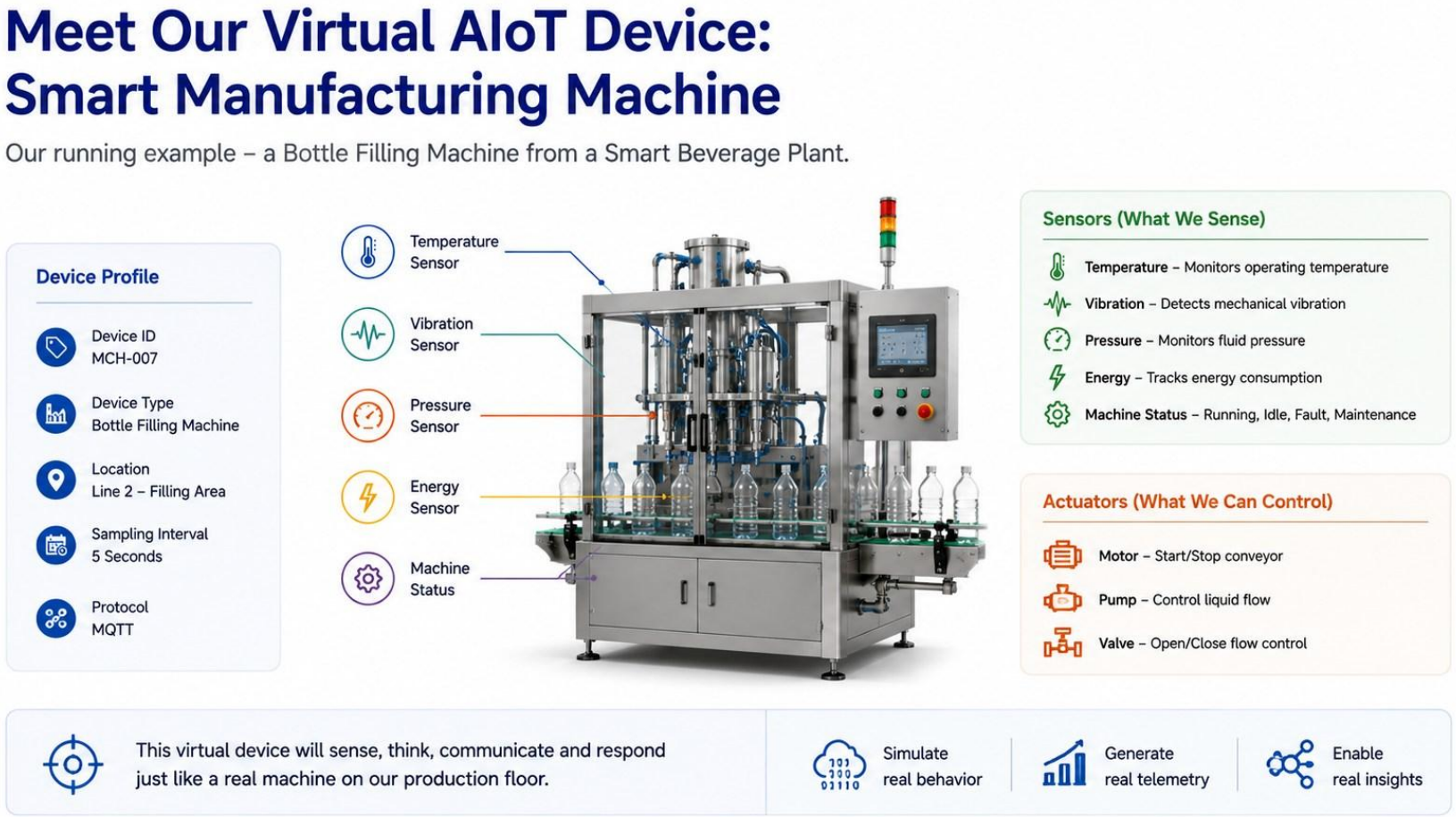
Simulateur Python, connexion MQTT et persistance des données

# Et si on pouvait construire un dispositif IoT sans matériel ?




*Créer, tester et faire évoluer des dispositifs connectés intelligents — entièrement en logiciel.*

# Exemple : une machine de remplissage de bouteilles




*Capteurs (température, vibration, pression, énergie) et actionneurs (moteur, pompe, vanne) du dispositif MCH-007.*

# Pourquoi simuler un dispositif IoT en logiciel ?

1. **Prototypage rapide**
   Valider les idées avant que le matériel ne soit disponible.
2. **Tests approfondis**
   Applications, intégrations et comportements en environnement contrôlé.
3. **Réduction des risques**
   Simuler pannes, anomalies et cas limites en toute sécurité.
4. **Simulation à l'échelle**
   Des milliers de dispositifs pour tester scalabilité et performance.
5. **Génération de données**
   Télémétrie réaliste à haute fréquence pour l'analytique.
6. **Apprentissage & innovation**
   Expérimenter sans risque, accélérer l'innovation.

*Créer, tester et faire évoluer des dispositifs connectés intelligents — entièrement en logiciel.*

# Cycle de comportement du dispositif : Sense → Process → Decide → Communicate

1

**1. Sense**

Collecte des données des capteurs : température, vibration, pression, énergie.

→

2

**2. Process**

Validation, filtrage du bruit, calcul de métriques, maintien de l'état.

→

3

**3. Decide**

Détection de conditions : normal, warning, fault, maintenance.

→

4

**4. Communicate**

Publication de la télémétrie via MQTT, réception de commandes.

*Boucle de feedback continue — c'est ainsi qu'un vrai dispositif IoT se comporte sur le terrain.*

# Identité et configuration du dispositif

Chaque dispositif IoT doit avoir une identité unique et une configuration claire.

```
import time
from dataclasses import dataclass, field

@dataclass
class DeviceConfig:
    device_id: str = "MCH-007"
    device_type: str = "Bottle Filling Machine"
    location: str = "Production Line 2"
    sampling_interval: int = 5  # secondes
    status: str = "RUNNING"
    created_at: float = field(
        default_factory=time.time)

# Instance de configuration
config = DeviceConfig()
```

**Identification unique**
Distingue ce dispositif parmi des milliers d'autres.

**Conscience du contexte**
Localisation et type utiles au routage et à l'analytique.

**Comportement cohérent**
Intervalle d'échantillonnage et configuration prévisibles.

**Traçabilité**
Horodatage de création pour audit et dépannage.

# Simuler les capteurs virtuels

Quatre capteurs génèrent des valeurs réalistes et légèrement aléatoires.

```
import random
from typing import Dict

class VirtualSensors:
    def read_temperature(self) -> float:
        return round(random.uniform(70.0, 85.0), 2)

    def read_vibration(self) -> float:
        return round(random.uniform(1.5, 3.5), 2)

    def read_pressure(self) -> float:
        return round(random.uniform(5.0, 7.0), 2)

    def read_energy(self) -> float:
        return round(random.uniform(1.0, 2.0), 2)

    def read_all(self) -> Dict[str, float]:
        return {
            "temperature": self.read_temperature(),
            "vibration": self.read_vibration(),
            "pressure": self.read_pressure(),
            "energy": self.read_energy(),
        }
```

**Plages réalistes**
Valeurs générées dans des plages opérationnelles réelles.

**Capteurs indépendants**
Chaque capteur simule sa propre lecture.

**Conception extensible**
Facile d'ajouter d'autres capteurs plus tard.

# Publier la télémétrie vers MQTT

Le simulateur convertit la télémétrie en JSON et la publie sur le broker.

```
import json
import paho.mqtt.client as mqtt

TOPIC = "factory/production-line-2/machine/MCH-007/telemetry"

def publish_telemetry(client, telemetry):
    payload = json.dumps(telemetry)
    result = client.publish(TOPIC, payload, qos=1)
    if result.rc == 0:
        print(f"Publié sur {TOPIC}")
    else:
        print(f"Échec: {result.rc}")

client = mqtt.Client(client_id="MCH-007-Device")
client.connect("localhost", 1883, 60)
publish_telemetry(client, telemetry_data)
```

**Découplage**
MQTT découple le dispositif des applications qui consomment les données.

**Topic structuré**
factory/ligne/machine/id/telemetry — hiérarchie claire et filtrable.

**QoS choisi**
QoS 1 pour garantir la livraison de la télémétrie.

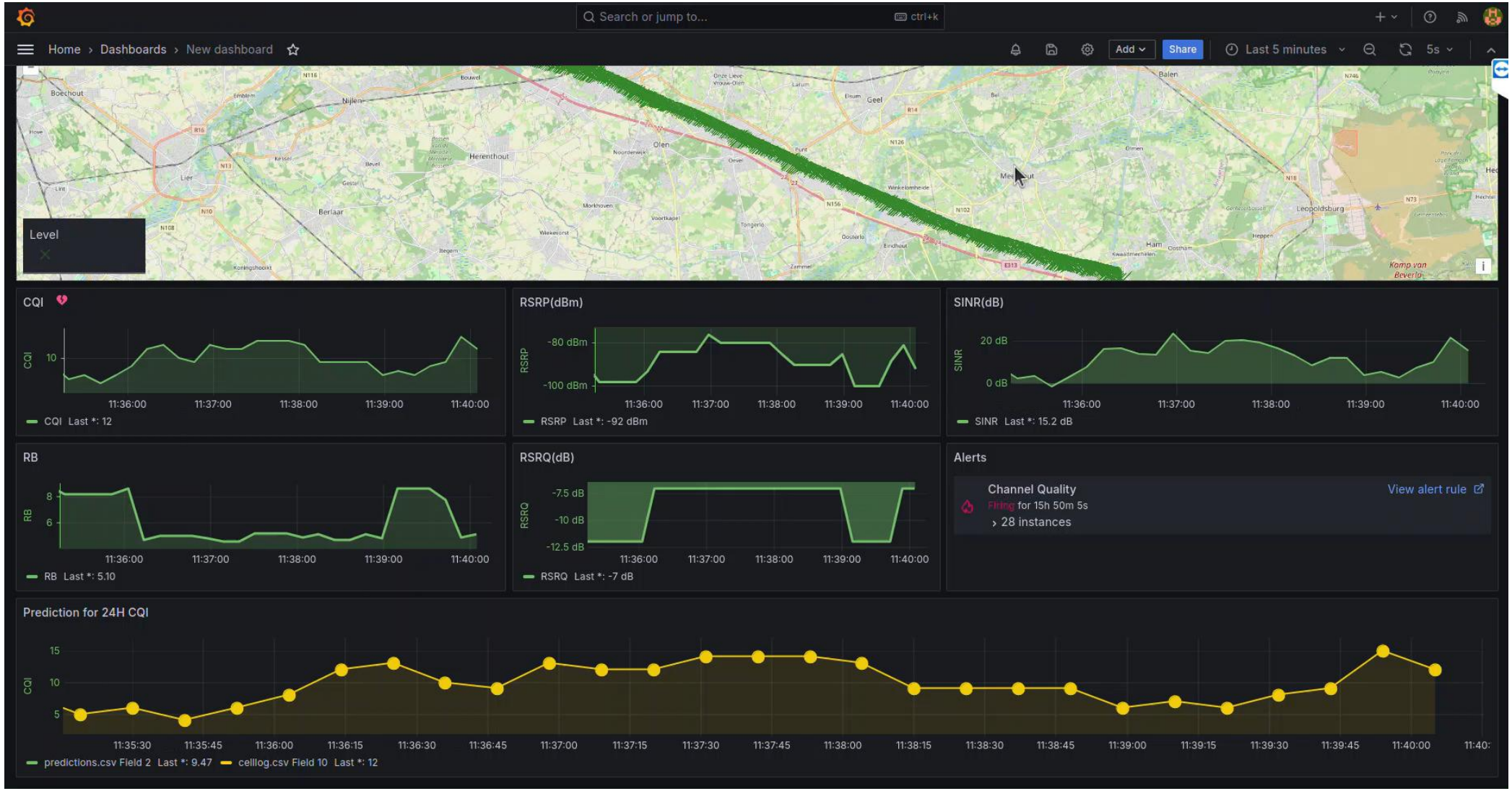
SYNTHÈSE

# De la simulation au composant AIoT d'entreprise

- Identité & configuration → base du dispositif
- Capteurs virtuels → génèrent une télémétrie réaliste
- Télémétrie structurée (JSON) → format standard IoT
- Boucle continue → sense, process, communicate
- Connexion MQTT → publication fiable de la télémétrie
- Persistance MongoDB → mémoire et historique du dispositif

**→ Étape suivante : stocker la télémétrie dans MongoDB pour analyser les tendances (voir TP5 de la fascicule).**

# Grafana un outil de supervision professionelle



MODULE 6

# Cloud AWS pour l'IoT

IoT Core, DynamoDB, traitement Stream et Batch

# Cycle de vie de la donnée IoT

1

**Collecte**

Capture des données par les capteurs et dispositifs.

→

2

**Transmission**

Acheminement via le réseau vers le cloud/edge.

→

3

**Stockage**

Persistance pour usage ultérieur.

→

4

**Traitement**

Temps réel (stream) ou différé (batch).

→

5

**Visualisation**

Rapports et tableaux de bord métier.

# AWS IoT Core : composants clés

1 **Thing**

Brique de base : un dispositif final générant des données via un ou plusieurs capteurs.

2 **Device Registry**

Enregistrement et provisionnement en masse des dispositifs, avec hiérarchies.

3 **Device Shadow**

Carte d'état persistante du dispositif — accessible même hors connexion.

4 **Rule Engine**

Achemine automatiquement les données vers DynamoDB, S3, Lambda, Kinesis… via une syntaxe SQL.

5 **Jobs Management**

Déploiement de mises à jour, reboot, certificats à grande échelle.

6 **Sécurité**

Certificats X.509, MQTTS, HTTPS pour authentification et communication chiffrée.

# AWS DynamoDB : base NoSQL managée pour l'IoT

## Caractéristiques

- Entièrement managée et serverless — zéro administration
- Auto-scalable selon la charge de lecture/écriture
- Redondante — multi-zone et multi-région
- Très faible latence grâce aux hash maps
- Chiffrement au repos, sauvegarde/restauration

## Modèle de données

- Collection ≈ table (SQL)
- Item ≈ ligne — attributs imbriqués jusqu'à 32 niveaux
- Clé de partition + clé de tri pour un accès optimisé
- Ingestion facile depuis IoT Core
- Sortie vers Kinesis pour traitement en flux

# Traitement Stream vs Batch

| Critère | Stream Processing | Batch Processing |
|---|---|---|
| Timing | Données traitées en temps réel | Données collectées puis traitées ensemble |
| Délai de résultat | Secondes à minutes | Minutes, heures voire jours |
| Objectif | Alertes, insights immédiats | Tendances, corrélations, analyse historique |
| Puissance requise | Faible mais continue | Élevée mais sur courte durée |
| Coût | Peut être coûteux (disponibilité continue) | Généralement plus économique |

# AWS Kinesis : le streaming managé pour l'IoT

1. **Kinesis Data Streams**
   Shards = unité d'échelle. 1 shard : 1000 TPS en écriture / 5 TPS en lecture.

2. **Kinesis Firehose**
   Livraison sans code vers S3, Redshift, Elasticsearch, avec auto-scaling.

3. **Kinesis Pipes**
   Divise un flux en plusieurs chemins de traitement (Lambda, EMR, Redshift).

4. **AWS SQS**
   File de messages pour découpler les composants, avec délai programmable jusqu'à 15 min.

5. **Kafka**
   Système pub/sub open-source distribué, hautement fiable et scalable (ex. Netflix).

6. **Kinesis vs SQS**
   Kinesis pour le temps réel ; SQS pour des tâches en arrière-plan (jobs).

# Traitement Batch : outils et cas d'usage IoT

1 **Apache Spark**
Moteur unifié 100× plus rapide que MapReduce ; Java, Scala, Python, R, SQL.

2 **Stockage**
HDFS, HBase, DynamoDB, S3, Azure Blob.

3 **Reporting**
Tableau, Power BI, Amazon QuickSight.

4 **Mini-batch à l'Edge**
Agrégation locale, réduction du transfert de données montant.

5 **YARN**
Gestionnaire de cluster négociant les ressources pour Spark.

6 **Avantages / défis**
Efficacité et simplicité, mais dépannage complexe et coûts d'initialisation élevés.

ÉTUDE DE CAS

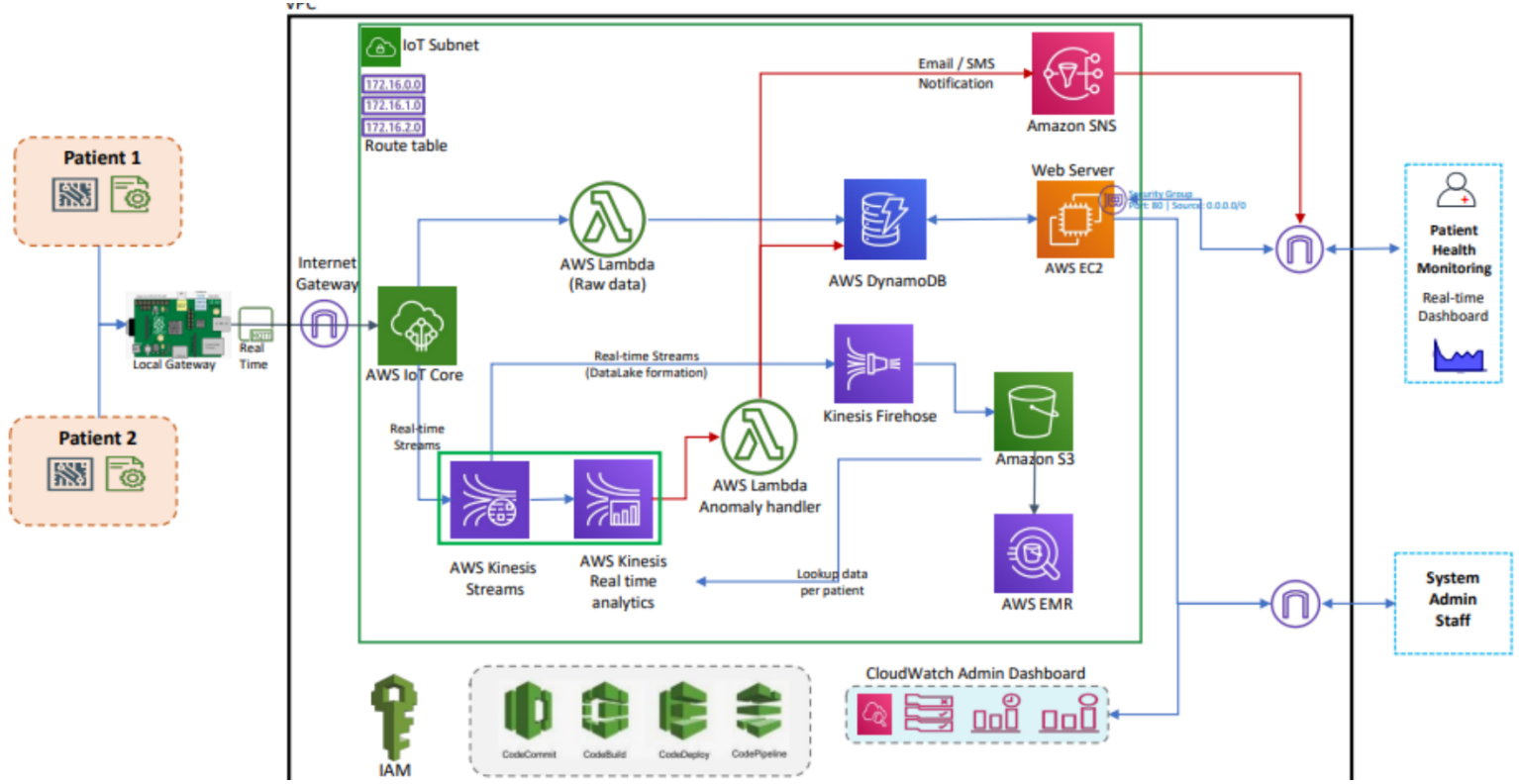
# Cas d'usage : santé connectée avec AWS IoT

*Surveillance en temps réel de patients via capteurs connectés*

- AWS IoT Core reçoit la télémétrie des capteurs patients via une passerelle locale
- AWS Lambda traite les données brutes et les anomalies en temps réel
- DynamoDB stocke les données patient pour un accès à faible latence
- Kinesis Data Streams + Analytics détectent les anomalies en continu
- Amazon SNS envoie des notifications email/SMS en cas d'alerte
- Amazon S3 + EMR permettent l'analyse approfondie et le data lake

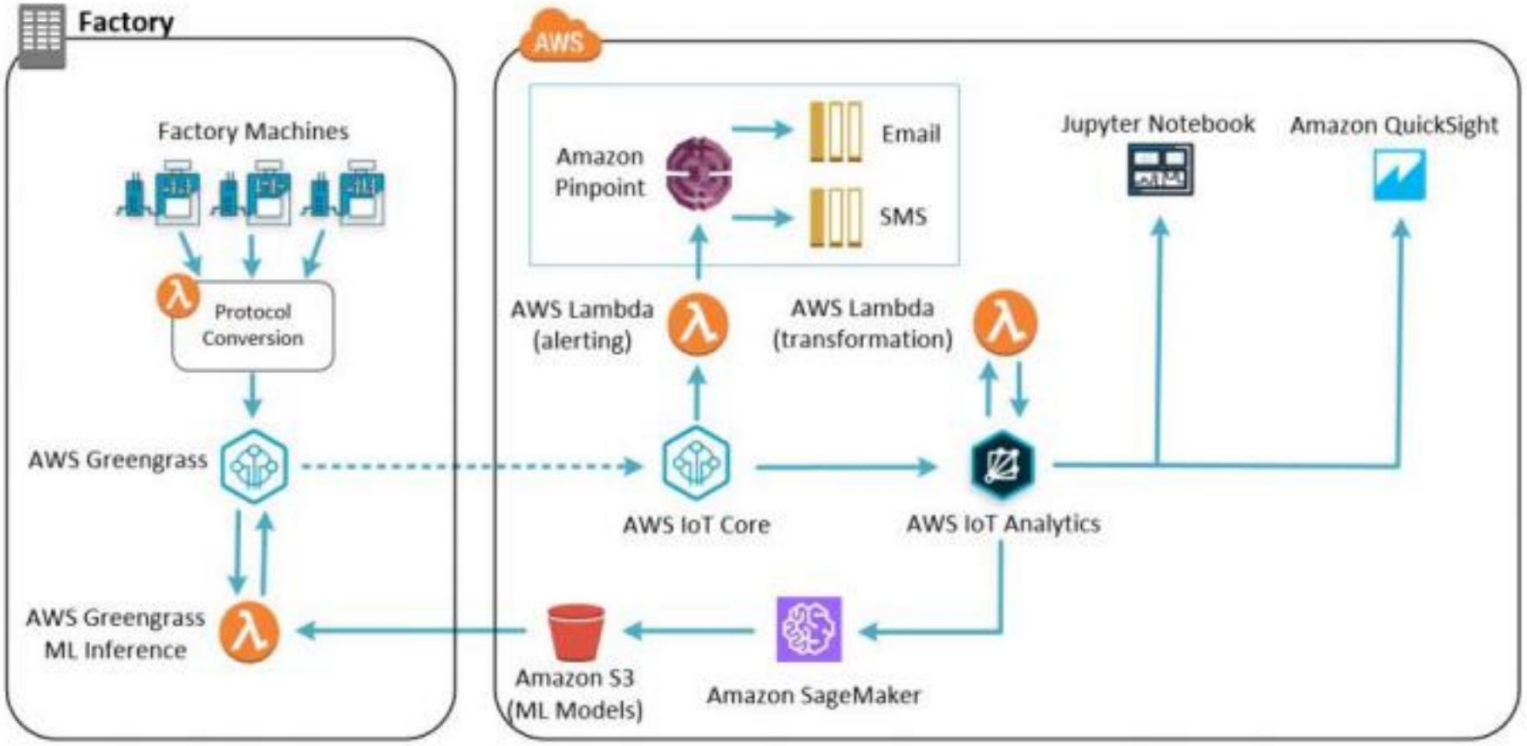
**Résultat : un tableau de bord de surveillance patient en temps réel, avec alertes automatiques.**

# Architecture réelle : santé connectée sur AWS




*IoT Core → Lambda/DynamoDB pour l'accès temps réel · Kinesis → S3/EMR pour l'analyse approfondie.*

# Architecture réelle : usine intelligente avec AWS Greengrass




*Inférence ML en périphérie (Greengrass) alimentée par des modèles entraînés dans le cloud (SageMaker).*

MODULE 7

# IA & Machine Learning pour l'AIoT

Du concept au modèle entraîné et déployé

# Intelligence Artificielle, Machine Learning, Modèle

## IA — l'objectif global

- Champ visant à construire des systèmes capables de tâches qui requièrent normalement l'intelligence humaine
- Raisonnement, apprentissage, décision, reconnaissance vocale, compréhension visuelle
- Exemples : IA générative (ChatGPT), vision par ordinateur, NLP, recommandation

## Machine Learning — la méthode

- Sous-domaine de l'IA : les systèmes apprennent à partir des données sans programmation explicite
- Modèle = Algorithme + Données + Entraînement
- Le modèle est le moteur réel qui alimente l'application IA
- Apprend en reconnaissant motifs et relations dans les données

# Programmation traditionnelle vs Intelligence Artificielle

| Aspect | Programmation traditionnelle (règles) | IA / Machine Learning (apprentissage) |
|---|---|---|
| Logique | Règles fixes écrites explicitement | Le système apprend les motifs depuis les données |
| Adaptabilité | Nécessite une mise à jour des règles | S'améliore avec plus de données et d'entraînement |
| Exemple | "Si température > 40 → alerte" | Prédire une panne à partir de plusieurs capteurs |
| Flux | Données + Règles → Résultat | Données + Résultats attendus → Modèle entraîné → Prédiction |

# Workflow Machine Learning dans un système AIoT

1

**1. Collecte**

Capteurs génèrent température, vibration, pression, courant en continu.

→

2

**2. Prétraitement**

Nettoyage, valeurs manquantes, filtrage du bruit, normalisation.

→

3

**3. Feature Engineering**

Transformation en variables porteuses de sens (tendances, moyennes).

→

4

**4. Entraînement**

Le modèle apprend motifs et relations à partir des données historiques.

→

5

**5. Évaluation**

Mesure de la fiabilité (précision, rappel, F1) avant mise en production.

*Historique + Entraînement + Correction d'erreurs = Modèle de prédiction intelligent.*

# Feature Engineering : le pont entre données brutes et prédictions

*Transformer les données brutes en variables porteuses de sens que le modèle peut réellement exploiter.*

1

### Données brutes

Température, vibration, pression, courant moteur, horodatages.

2

### Variables dérivées

Variation de température, moyenne glissante 10 min, vitesse d'augmentation de pression.

3

### Indicateurs temporels

Heure du jour, jour de semaine, équipe de travail, week-end.

4

### Inférence obtenue

Certaines machines consomment plus d'énergie la nuit ; pannes plus fréquentes en heures de pointe.

5

### Principe clé

Un algorithme simple avec de bonnes features surpasse souvent un algorithme complexe avec des données de mauvaise qualité.

6

### Impact business

La réussite d'un projet IA dépend souvent plus du feature engineering que de la complexité du modèle.

# Apprentissage supervisé : 4 algorithmes clés en AIoT

| Algorithme | Principe | Cas d'usage AIoT |
|---|---|---|
| Régression Logistique | Prédit la probabilité d'un événement binaire | Prédiction de panne (oui/non), risque de défaillance |
| Arbre de Décision | Structure de décision basée sur des règles, facile à interpréter | Classification d'état machine : Normal → Warning → Fault |
| Random Forest | Ensemble d'arbres — meilleure précision, réduit le surapprentissage | Maintenance prédictive avec entrées capteurs multiples |
| SVM (Support Vector Machine) | Trouve la frontière optimale entre classes | Détection d'anomalies, comportement normal vs anormal |

*Choix selon la complexité, l'interprétabilité requise, la précision visée et les contraintes du dispositif.*

ÉTUDE DE CAS

# Cas d'usage : bâtiment intelligent — supervision énergie & équipements

*Capteurs IoT sur HVAC, température, consommation énergétique*

- Régression Logistique → Le compresseur va-t-il tomber en panne dans les 24h ? (alerte précoce)
- Arbre de Décision → Quel est l'état actuel du HVAC ? Normal → Surchargé → Défaut
- Random Forest → Comment optimiser la consommation énergétique sans perdre en confort ?
- SVM → Le comportement de l'équipement est-il anormal par rapport aux patterns habituels ?

**Impact business : moins de pannes imprévues, meilleure efficacité, coûts de maintenance réduits, sécurité accrue.**

# Apprentissage non supervisé : découvrir l'inconnu

*Identifie des motifs et structures dans les données sans étiquettes prédéfinies.*

1

### Pourquoi l'utiliser

Grands volumes de données non étiquetées, environnements dynamiques, pannes inconnues.

2

### K-Means

Regroupe les motifs similaires (clustering des comportements machine).

3

### DBSCAN

Détecte les clusters et le bruit — identifie les comportements atypiques.

4

### Isolation Forest

Isole les motifs anormaux — détection d'anomalies.

5

### Cas d'usage AIoT

Clustering d'équipements, découverte de patterns de consommation, segmentation de flotte.

6

### Bénéfice clé

Détecte les problèmes inconnus avant qu'ils ne deviennent des pannes coûteuses.

# Évaluer un modèle : les métriques essentielles

| Métrique | Signification | Importance en AIoT |
|---|---|---|
| Accuracy | Proportion globale de prédictions correctes | Performance générale du modèle |
| Precision | Fiabilité des alertes de panne | Réduit les fausses alertes |
| Recall | Capacité à détecter les vraies pannes | Évite les pannes manquées (impact sécurité) |
| F1 Score | Équilibre entre précision et rappel | Qualité de prédiction fiable |
| MAE | Erreur moyenne de prédiction | Utile pour le forecasting |

*Fausse alerte = maintenance inutile et coûts. Panne manquée = dommage matériel, perte de production, risque sécurité.*

MODULE 8

# Edge AI & TinyML

De l'intelligence cloud à l'intelligence embarquée

# Cloud AI vs Edge AI

| Caractéristique | Cloud AI | Edge AI |
|---|---|---|
| Latence | Élevée | Très faible |
| Connexion Internet | Requise | Non requise |
| Confidentialité | Plus faible (données transmises) | Plus élevée (données restent locales) |
| Coût | Continu (abonnement / usage) | Investissement ponctuel |
| Cas d'usage | Analytique complexe, entraînement de modèles | IoT, robotique, sécurité critique |

*En pratique : Edge pour la vitesse et l'autonomie, Cloud pour l'échelle et l'intelligence approfondie (approche hybride).*

# 4 avantages clés de l'Edge AI

1

### Latence réduite

Traitement en millisecondes, réponse instantanée pour la sécurité et l'automatisation.

2

### Confidentialité renforcée

Les données sensibles restent locales — réduit les risques et facilite la conformité (RGPD).

3

### Coûts de bande passante réduits

Seules les informations pertinentes sont envoyées au cloud.

4

### Fiabilité accrue

Les dispositifs continuent de fonctionner même en cas de perte de connectivité.

# TinyML : le Machine Learning sur microcontrôleurs

## Qu'est-ce que le TinyML ?

- Déploiement de modèles IA sur du matériel ultra contraint (microcontrôleurs, DSP)
- Fonctionne avec quelques kilo-octets de mémoire seulement
- Consommation énergétique de l'ordre du milliwatt
- Cas d'usage : mot de réveil vocal, reconnaissance de geste, détection d'anomalie

## Edge AI vs TinyML

- Edge AI : matériel plus puissant (smartphones, GPU, serveurs edge)
- TinyML : matériel ultra-contraint (microcontrôleurs < 1 Mo RAM)
- Edge AI vise la réduction de latence ; TinyML vise l'autonomie énergétique
- TinyML tourne souvent en bare-metal ou RTOS léger (pas d'OS complet)

# L'IA répartie sur 3 niveaux : exemple d'une montre santé connectée

1

### Tiny AI (montre)

Détection d'anomalie de rythme cardiaque, détection de chute — en continu, ultra basse consommation.

2

### Edge AI (mobile/gateway)

Analyse de motifs dans le temps, génération d'alertes et notifications.

3

### Cloud AI

Analytique prédictive de santé, analyse de tendances à long terme, recommandations personnalisées.

# Matériel Edge AI / TinyML : les briques clés

| Composant | Rôle | Exemples |
|---|---|---|
| NPU (Neural Processing Unit) | Accélère les opérations IA, faible latence et faible consommation | Arm Ethos, Apple Neural Engine |
| GPU | Traitement parallèle intensif — entraînement, vision avancée | NVIDIA Jetson series |
| VPU | Traitement d'image et vision par ordinateur | Intel Movidius Myriad X |
| TPU | ASIC dédié à l'accélération du Machine Learning | Google Coral Edge TPU |
| MCU (Microcontrôleur) | Ultra basse consommation, ressources limitées | STM32, ESP32, ATmega328P (Arduino) |

*Frameworks logiciels : TensorFlow Lite / TFLite Micro, ONNX Runtime, Edge Impulse, OpenCV.*

# Techniques d'optimisation de modèles pour l'Edge

| Technique | Principe | Exemple |
|---|---|---|
| Quantization | Conversion 32 bits → 8/16 bits : modèle plus petit, inférence plus rapide | Caméra intelligente détectant des intrus en 8 bits, sans cloud |
| Pruning | Suppression des poids/neurones inutiles pour réduire calcul et mémoire | Modèle de maintenance prédictive allégé sur passerelle industrielle |
| Knowledge Distillation | Un petit modèle "élève" apprend à imiter un grand modèle "professeur" | Wearable santé détectant un rythme cardiaque anormal en léger |
| Architectures optimisées | Conception dédiée à l'edge plutôt que compression d'un grand modèle | MobileNet pour la détection de visage sur smartphone/sonnette |

*Bénéfices combinés : mémoire réduite, inférence plus rapide, meilleure autonomie, empreinte de stockage réduite.*

ÉTUDE DE CAS

# Étude de cas : optimiser un modèle IA pour la sécurité en usine

*Détection du port du casque et des équipements de protection par caméras IA*

- Défis initiaux : latence cloud élevée, coûts de bande passante, vie privée des employés, passerelle limitée
- 1. Quantization : modèle converti en 8 bits → taille réduite de ~75%
- 2. Pruning : suppression des poids redondants → charge de calcul réduite
- 3. Knowledge Distillation : modèle "élève" plus compact, précision maintenue
- 4. Déploiement à l'edge : modèle optimisé sur une passerelle connectée aux caméras

**Résultats : détection < 100 ms, -80% de bande passante, confidentialité préservée, détection instantanée des violations de sécurité.**

MODULE 9

# Études de cas & synthèse

Applications réelles et vision d'ensemble de l'architecture AIoT

CAS RÉEL

# Internet of Vehicles (IoV) — Volkswagen

*MQTT à très grande échelle pour la connectivité automobile*

- Intégration IT dans les véhicules : communication véhicule ↔ infrastructure ↔ cloud
- Mises à jour trafic en temps réel, analytique prédictive, services personnalisés
- Conduite autonome et gestion de flotte pour améliorer l'expérience conducteur
- MQTT choisi pour sa haute concurrence et sa faible latence entre véhicules
- EMQ Technologies (broker MQTT open-source) utilisé pour une communication sécurisée et performante

CAS RÉEL

# FleetLINK — Gestion de flotte connectée

- Solution de gestion de transport sans fil : suivi GPS temps réel, rapports automatisés, alertes
- Contrôle bidirectionnel complet des unités à distance
- Verrouillage géolocalisé programmable pour sécuriser les remorques
- Protocoles : cellulaire, MQTT, Wi-Fi
- Données collectées : consommation carburant, coûts de maintenance, localisation, coûts main-d'œuvre

# Panorama de cas d'usage IoT dans le monde

1

### ioFog — Edge Computing Framework

Framework open-source de microservices conteneurisés pour l'edge computing.

2

### Novetta — Réponse aux catastrophes

LoRa + AWS Snowball Edge pour localiser personnes, véhicules et équipements sans réseau cloud.

3

### Agriculture de précision

Caméras + ML embarqué (AWS Greengrass) pour suivre la croissance des cultures sans coût 3G continu.

4

### DB Railway (Allemagne)

Suivi des trains en temps réel via MQTT (Eclipse Mosquitto/Paho) toutes les 10 secondes.

5

### Smart Electricity Grid

Réseau électrique dynamique réagissant à la consommation réelle en temps réel.

6

### uReplicator (Uber)

Réplication Kafka multi-datacenter fiable pour les logs temps réel et batch à très grande échelle.

# L'architecture AIoT complète, de bout en bout

**1. Dispositif (Device)**

Capteurs, actionneurs, simulateur Python, identité & télémétrie

**2. Connectivité**

Réseau + MQTT (Publish/Subscribe, QoS, sécurité TLS)

**3. Edge**

Traitement local, filtrage, inférence Edge AI / TinyML

**4. Cloud**

IoT Core, DynamoDB, Kinesis (stream), Spark (batch)

**5. Intelligence**

Machine Learning : supervisé, non supervisé, modèles optimisés

- Chaque couche du cours correspond à une brique de cette architecture.
- Le dispositif capte, le réseau transporte, l'edge décide vite,
- le cloud stocke et analyse à grande échelle, l'IA rend le système intelligent.

SYNTHÈSE

# Ce qu'il faut retenir de ce cours

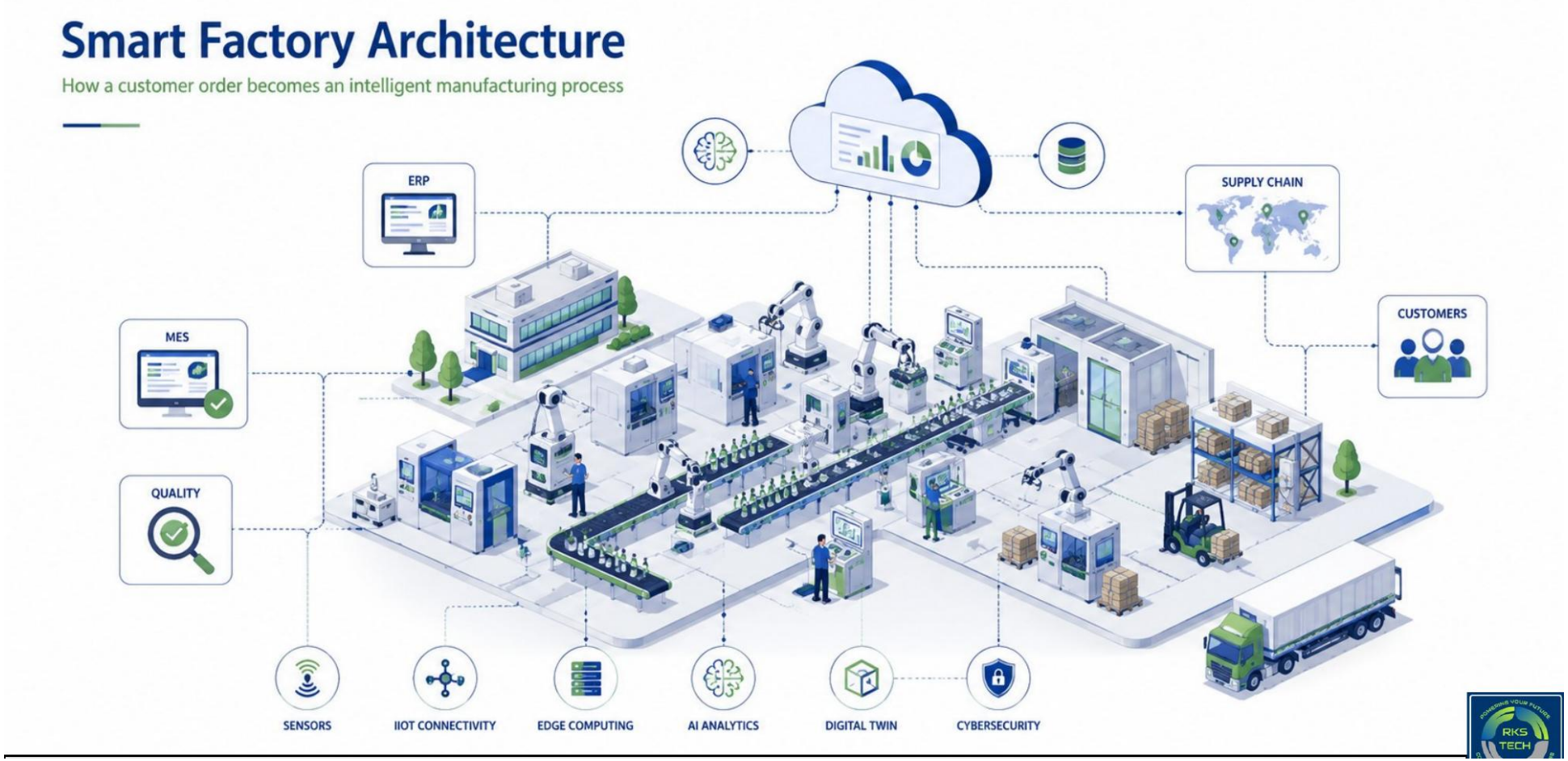
- L'IoT relie capteurs, actionneurs et backend via un réseau adapté
- MQTT est le standard de messagerie pour l'IoT d'entreprise
- Cloud et Edge sont complémentaires, pas concurrents
- Un dispositif AIoT peut être entièrement prototypé en Python
- AWS IoT Core + DynamoDB + Kinesis forment une chaîne cloud complète
- Le Machine Learning transforme la donnée capteur en décision intelligente
- L'Edge AI et le TinyML amènent l'intelligence au plus près du capteur
- La sécurité (authentification, chiffrement, autorisation) est non négociable

MODULE 10

# Architecture d'entreprise & Jumeaux numériques

Concevoir des systèmes AIoT scalables, du device à l'entreprise

# L'usine intelligente : de la commande client au produit fini




*ERP, MES, capteurs, robots, edge et cloud AI orchestrés autour d'une même ligne de production.*

# Les 6 défis de conception de l'IoT d'entreprise

| Défi | Enjeu | Principe directeur |
|---|---|---|
| 1. Scalabilité | Passer de 10 à des millions de dispositifs sans re-concevoir | Concevoir pour la croissance, dès le départ |
| 2. Interopérabilité | Connecter des protocoles hétérogènes (MQTT, OPC UA, Modbus…) | Unifier via une plateforme AIoT commune |
| 3. Sécurité & vie privée | Une brèche à n'importe quelle couche impacte tout le système | Sécurité par conception, à chaque couche |
| 4. Latence & Edge | Toute décision ne doit pas transiter par le cloud | Edge pour le temps réel, Cloud pour l'analyse |
| 5. Données & IA | Une IA n'est fiable que si les données le sont | Une gouvernance de donnée solide avant l'IA |
| 6. Fiabilité & résilience | Le système doit fonctionner même en cas de panne partielle | La résilience est une exigence, pas une option |

# Les briques d'une architecture de référence d'entreprise

**Couche Device & Protocole**

Capteurs, passerelles, protocoles de terrain

**Couche Edge**

Traitement local, filtrage, décisions temps réel

**Couche Données**

Ingestion, stockage structuré/non structuré, gouvernance

**Couche IA & Analytique**

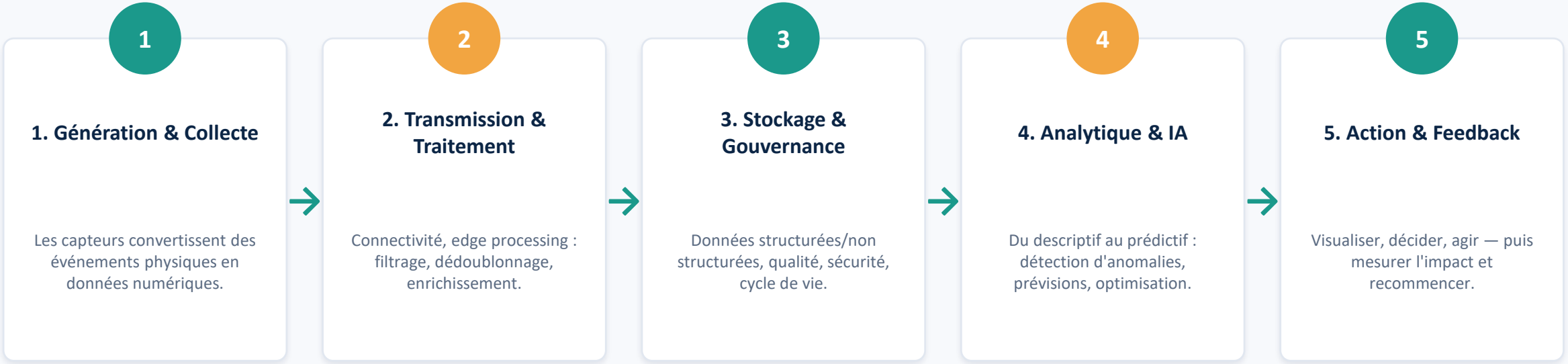
Machine Learning, prédiction, optimisation

**Couche Intégration & Application**

APIs, tableaux de bord, systèmes d'entreprise (ERP/MES)

- Une architecture réutilisable : construite une fois, déployée partout.
- Principes directeurs : réutilisable, scalable, sécurisée,
- interopérable, observable et intelligente.

# Le cycle de vie de la donnée IoT : une boucle continue




*Chaque étape du cycle génère de nouvelles données, alimentant une amélioration continue.*

# Qu'est-ce qu'un Jumeau Numérique (Digital Twin) ?

*Une représentation numérique vivante et synchronisée en temps réel d'un actif ou système physique, utilisée pour analyser, prédire et optimiser.*

1

### Convergence IoT + IA + Cyber-physique

Le jumeau numérique naît de la fusion de ces trois technologies.

2

### Pipeline continu

Actif physique → capteurs/connectivité → edge → plateforme cloud → IA & analytique → actions.

3

### Prédiction & simulation

Maintenance prédictive, simulation de scénarios, optimisation continue.

4

### Impact métier mesurable

Fabrication : -20 à 40% de temps d'arrêt. Énergie : -10 à 30% d'économies.

5

### Défis techniques

Qualité des données, complexité d'intégration, cybersécurité, dérive du modèle.

6

### Vision future

Jumeaux numériques intelligents, autonomes et centrés sur l'humain (Industrie 5.0).

# Jumeau numérique : de l'usine physique à sa réplique intelligente




*Capteurs IoT + IA analytique + systèmes cyber-physiques → visibilité temps réel, prédiction, simulation.*

ÉTUDE DE CAS

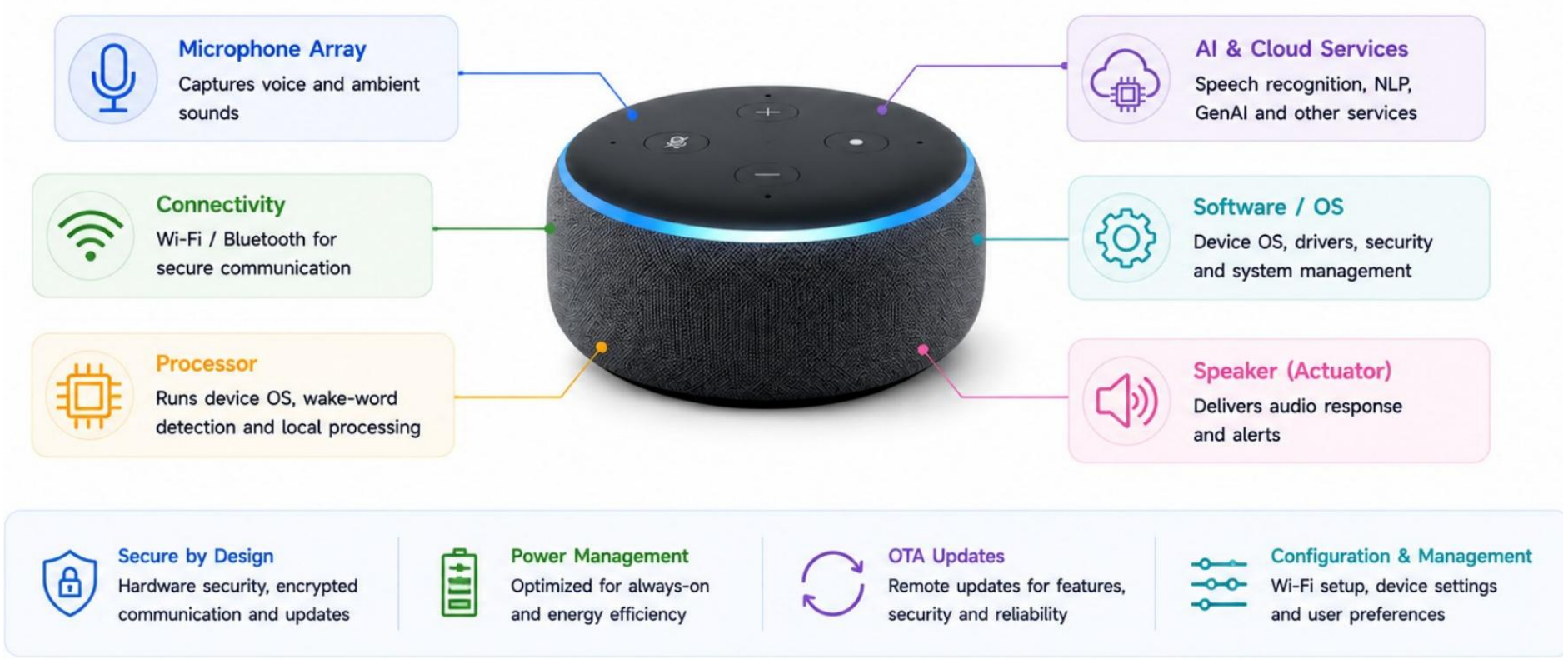
# Anatomie d'un dispositif intelligent : l'enceinte connectée

*Comment une enceinte intelligente reflète l'architecture AIoT complète*

- Couche Device → Microphone : capture la voix et le son ambiant
- Couche Connectivité → Wi-Fi / Bluetooth : communication sécurisée
- Couche Edge → Moteur de détection du mot de réveil ("Alexa") en local
- Couche Cloud → Service vocal cloud : traitement de la parole et des intentions
- Couche IA & Analytique → NLP et IA générative pour comprendre et répondre
- Couche Actionneur → Haut-parleur : délivre la réponse audio à l'utilisateur

**Un dispositif intelligent combine détection, connectivité, intelligence en périphérie, cloud IA et services métier.**

# Les composants d'une enceinte connectée




*Micro, connectivité, processeur, IA cloud, logiciel et haut-parleur — les briques d'un dispositif intelligent grand public.*

# Étude de cas : l'évolution du suivi d'actifs (Asset Tracking)

| Technologie | Couverture | Précision | Cas d'usage typique |
|---|---|---|---|
| Code-barres / QR | Intérieur uniquement | Manuel (ponctuel) | Point de vente, inventaire, suivi documentaire |
| Bluetooth Low Energy | Intérieur, extérieur limité | Moyenne | Entrepôts, campus, suivi de visiteurs |
| Wi-Fi | Intérieur, extérieur limité | Moyenne | Suivi d'actifs en intérieur |
| GPS | Extérieur uniquement | Élevée | Véhicules, flotte, conteneurs |
| UWB (Ultra-Wideband) | Intérieur, extérieur limité | Très élevée | Actifs de grande valeur en usine |
| Hybride (multi-tech) | Intérieur & extérieur | Élevée | Grandes entreprises, environnements complexes |

*Évolution : ère manuelle (papier, erreurs) → ère IoT (visibilité temps réel) → ère AIoT (prédiction et action autonome).*

# Le suivi d'actifs en action

From visibility to intelligence –
manage every asset, anywhere, in real time.




**Real-time Visibility**
Know location, status and movement instantly

**Smart Alerts**
Get notified on delays, deviations and exceptions

**Complete Traceability**
End-to-end chain of custody for compliance and trust

**Intelligent Insights**
AI-powered analytics for better decisions

**Operational Efficiency**
Reduce loss, optimize utilization and lower costs

*Visibilité temps réel, alertes intelligentes, traçabilité complète — de la donnée à l'action.*

# Panorama des cas d'usage AIoT d'entreprise par secteur

1

### Industrie 4.0 (Manufacturing)

Maintenance prédictive, inspection qualité par IA, jumeau numérique d'usine.

2

### Chaîne logistique intelligente

Visibilité temps réel, optimisation de flotte, gestion de température (chaîne du froid).

3

### Santé connectée

Surveillance à distance des patients, dispositifs médicaux connectés, soins proactifs.

4

### Villes & infrastructures intelligentes

Trafic, éclairage public, qualité de l'air, gestion des déchets, sécurité publique.

5

### Énergie, utilities & agriculture

Réseaux intelligents, monitoring des cultures, agriculture de précision.

6

### Bâtiments intelligents

Gestion énergétique, occupation, confort et sécurité optimisés en continu.

CAS RÉEL

# Étude de cas : Singapour — l'AIoT à l'échelle d'une nation

*Une même architecture d'entreprise, déployée à l'échelle nationale*

- Smart Nation Sensor Platform (SNSP) : système nerveux numérique connectant les données de toute la ville
- Transport intelligent (ITS) : trafic en temps réel, gestion du stationnement, information voyageurs
- Gestion intelligente des déchets, de l'énergie et de la sécurité publique
- Virtual Singapore : jumeau numérique 3D de la nation pour la planification urbaine et la gestion des risques
- Résultat : une nation plus intelligente, résiliente et durable grâce à une architecture AIoT unifiée

# Singapour : l'AIoT à l'échelle d'une nation



*Des dispositifs individuels aux solutions d'entreprise, jusqu'à une nation entière — Enterprise AIoT à grande échelle.*

MODULE 11

# Sécurité, vie privée & éthique de l'IA

Construire un AIoT digne de confiance

# Pourquoi l'AIoT exige un nouveau réflexe sécurité

*L'AIoT ne se limite pas à des dispositifs connectés : l'intelligence, l'analytique et l'action autonome changent la donne.*

1

### Surface d'attaque élargie

Des milliards de dispositifs et de données en mouvement à travers tout l'écosystème.

2

### Modèles qui apprennent et s'adaptent

Comportement parfois imprévisible, plus difficile à auditer qu'une règle fixe.

3

### Manipulation des décisions

Un attaquant peut chercher à influencer les décisions prises par l'IA elle-même.

4

### Risque physique ET numérique

Une compromission peut avoir un impact dans le monde réel, pas seulement informatique.

5

### Gouvernance requise

Sécurité, éthique et gouvernance doivent travailler ensemble, pas en silos.

6

### Chaque couche = cible potentielle

Dispositifs, edge, modèles IA, données, pipeline, applications — tout est exposé.

# Les modèles IA : des cibles de choix

| Type d'attaque | Principe | Conséquence |
|---|---|---|
| Entrées adverses | Données subtilement modifiées pour induire le modèle en erreur | Décisions incorrectes ou dangereuses |
| Empoisonnement de données | Données d'entraînement corrompues intentionnellement | Comportement du modèle biaisé ou détourné |
| Vol / extraction de modèle | Reconstruction du modèle par requêtes répétées | Perte de propriété intellectuelle |
| Deepfakes & entrées synthétiques | Contenus générés pour tromper le système | Fraude, désinformation, usurpation |
| Injection de prompt | Instructions malveillantes cachées dans les entrées (IA générative) | Actions non souhaitées, fuite de données |

*La sécurité doit être intégrée à chaque étape du cycle de vie du modèle : collecte → entraînement → validation → déploiement → inférence → supervision → retrait.*

# Protéger la vie privée dans les systèmes pilotés par capteurs

*Chaque type de donnée capturée mérite une protection adaptée à sa sensibilité.*

1

### Données personnelles & comportementales

Identité, biométrie, habitudes — sensibilité la plus élevée.

2

### Données de localisation

GPS, géofencing — peuvent révéler des habitudes de vie.

3

### Privacy by Design

Protection intégrée dès la conception, pas ajoutée après coup.

4

### Techniques de préservation

Apprentissage fédéré, confidentialité différentielle, anonymisation, chiffrement.

5

### Gestion responsable

Collecter moins, protéger davantage, ne conserver que l'essentiel.

6

### Risques sur tout le cycle

Collecte → transmission → stockage → traitement → partage → suppression : chaque étape compte.

# Principes d'une IA responsable

| Principe | Définition |
|---|---|
| Équité (Fairness) | Les décisions du système IA traitent équitablement tous les individus, sans biais. |
| Transparence | Il est clair pour les utilisateurs comment et pourquoi le système prend ses décisions. |
| Explicabilité | Les décisions de l'IA peuvent être comprises par des humains, pas seulement exécutées. |
| Respect de la vie privée | Les données personnelles et les droits individuels sont protégés à chaque étape. |
| Redevabilité (Accountability) | Les organisations restent responsables des résultats et actions de leurs systèmes IA. |
| Supervision humaine | L'humain garde la capacité d'intervenir, corriger ou arrêter le système si nécessaire. |

# La pyramide de l'AIoT de confiance

**6. IA Éthique**

Équitable, transparente, centrée sur l'humain — le sommet de la confiance

**5. Conformité & Gouvernance**

Respect des réglementations et standards de gouvernance

**4. Vie privée**

Protection des données personnelles et des droits individuels

**3. Modèles IA & Données**

Intégrité, protection contre le vol et la manipulation

**2. Dispositifs & Edge de confiance**

Identité fiable, communication sécurisée

**1. Fondation Sécurité**

Protection des systèmes, réseaux et données — la base de tout

- Chaque couche s'appuie sur la précédente.
- Sécuriser l'AIoT n'est pas qu'une exigence technique :
- c'est un impératif business et de confiance.

MODULE 12

# Blockchain pour l'IoT d'entreprise

Confiance, traçabilité et automatisation entre organisations

# Pourquoi la Blockchain pour l'IoT d'entreprise ?

## Le problème des systèmes centralisés

- Autorité et propriété des données limitées à une seule organisation
- Risque de falsification des données
- Base de données centrale = point unique de défaillance
- Transparence limitée entre partenaires
- Réconciliation manuelle coûteuse entre organisations

## Ce que la Blockchain apporte

- Registre partagé (Shared Ledger) entre tous les participants
- Enregistrements immuables — infalsifiables une fois inscrits
- Confiance distribuée par consensus, sans autorité unique
- Transparence : visibilité de bout en bout des événements
- Smart Contracts : accords qui s'exécutent automatiquement

*La blockchain ne remplace pas l'IoT ni le cloud — elle renforce la confiance, l'immuabilité et la transparence entre organisations.*

# Architecture de référence IoT + Blockchain

**Dispositifs & Capteurs (Edge)**

Génèrent les données et événements du monde physique

**Plateforme IoT (Cloud)**

Collecte, agrège et traite les données à grande échelle

**Couche Blockchain (Trust Layer)**

Enregistre les événements métier critiques, garantit l'intégrité

**Applications & Services**

Tableaux de bord, systèmes d'entreprise, partenaires autorisés

- Règle d'or : le volume de données à haute fréquence reste hors chaîne (off-chain).
- Seuls les événements métier importants sont inscrits sur la blockchain (on-chain).
- Cela préserve la performance tout en garantissant la confiance.

# On-Chain vs Off-Chain : que stocker où ?

## Stockage Off-Chain (hors chaîne)

- Données volumineuses : vidéos, images, télémétrie brute
- Historique des capteurs, journaux détaillés
- Stockage cloud/edge classique — rapide et économique
- Seule une référence (hash) est ancrée sur la blockchain

## Stockage On-Chain (sur la chaîne)

- Événements métier importants (transferts, certifications)
- Preuves d'intégrité, empreintes (hash) des données
- Identité des dispositifs et enregistrements d'accès
- Exécution et résultats des Smart Contracts

*Principe directeur : stocker intelligemment, sur la chaîne uniquement ce qui exige preuve et confiance.*

# Identité décentralisée des dispositifs (DID)

*Chaque dispositif a besoin d'une identité numérique unique et vérifiable, sans dépendre d'une autorité centrale unique.*

1

### Problème de l'identité centralisée

Point unique de défaillance, dépendance fournisseur, gestion complexe à l'échelle.

2

### DID = identité auto-souveraine

Contrôlée par le dispositif ou l'organisation, ancrée sur la blockchain.

3

### Cycle de vie de l'identité

Fabrication → provisionnement → opération → révocation, chaque étape tracée.

4

### Onboarding sans contact

Provisionnement sécurisé et automatisé des nouveaux dispositifs.

5

### Bénéfices

Sécurité renforcée, résistance à la falsification, interopérabilité inter-organisations.

6

### Cas d'usage

Industrie, santé, chaîne logistique, véhicules connectés, énergie.

# Smart Contracts en action : maintenance prédictive automatisée

1

### 1. Condition détectée

Un capteur IoT détecte une anomalie (vibration, température).

→

2

### 2. Transaction blockchain

L'événement est soumis et vérifié par le réseau blockchain.

→

3

### 3. Smart Contract exécuté

Le contrat évalue les règles et conditions prédéfinies.

→

4

### 4. Action déclenchée

Ticket de maintenance créé, technicien notifié, ERP mis à jour.

*Résultat : actions automatiques, traçables et infalsifiables — sans intervention humaine manuelle.*

ÉTUDE DE CAS

# Traçabilité de la chaîne d'approvisionnement

*Du fournisseur au client, chaque événement capturé et vérifié sur la blockchain*

- Parcours produit de bout en bout : matière première → fabrication → contrôle qualité → emballage → transport → entrepôt → point de vente → client
- Chaque étape est horodatée et enregistrée de manière immuable et infalsifiable
- Smart Contracts automatisent les paiements, alertes de température et mises à jour de stock
- Secteurs concernés : pharmaceutique (chaîne du froid), agroalimentaire, aérospatiale, automobile, produits de luxe

**Bénéfices : prévention de la contrefaçon, visibilité de bout en bout, conformité réglementaire, confiance client renforcée.**

# Panorama des cas d'usage industriels de la Blockchain

1

### Manufacturing & automatisation industrielle

Identité des machines, traçabilité de production, ordres de travail numériques.

2

### Énergie & Utilities

Réseaux intelligents, certificats d'énergie renouvelable, échange pair-à-pair, recharge VE.

3

### Santé & sciences de la vie

Identité des dispositifs médicaux, traçabilité des médicaments, dossiers de santé électroniques.

4

### Villes & infrastructures connectées

Gestion du trafic, stationnement intelligent, services aux citoyens, infrastructures publiques.

5

### Gestion d'actifs & jumeaux numériques

Passeports d'équipement, propriété d'actifs, maintenance prédictive partagée.

6

### Valeur métier commune

Confiance améliorée, fraude réduite, collaboration sécurisée, automatisation plus rapide.

SYNTHÈSE

# Blockchain + IoT + Identité décentralisée + Smart Contracts

- La blockchain devient la couche de confiance de l'Industrie 4.0
- L'IoT génère la donnée ; la blockchain la sécurise et la rend fiable
- Identité décentralisée : chaque dispositif a une identité auto-souveraine
- Les Smart Contracts automatisent les workflows métier inter-organisations
- On-chain pour la confiance, off-chain pour le volume — un principe clé
- Ensemble : IoT, IA, Digital Twins, Blockchain construisent l'entreprise de confiance

MODULE 13

# Sécurité avancée & conformité de l'IoT d'entreprise

Zero Trust, identité des dispositifs, mises à jour OTA et normes de sécurité

# Zero Trust pour l'IoT d'entreprise : les 5 principes fondamentaux

*« Never Trust, Always Verify » — ne jamais faire confiance par défaut, toujours vérifier.*

1

### 1. Never Trust

Aucun utilisateur, appareil ou application n'est digne de confiance par défaut, où qu'il se trouve.

2

### 2. Always Verify

Authentifier et vérifier chaque demande d'accès en continu, avec identité forte du device.

3

### 3. Least Privilege

N'accorder que le minimum d'accès requis pour accomplir une tâche donnée, rien de plus.

4

### 4. Assume Breach

Concevoir comme si une brèche était déjà survenue : limiter l'impact et le mouvement latéral.

5

### 5. Continuous Monitoring

Surveiller en continu dispositifs, applications et réseaux pour détecter les anomalies en temps réel.

6

### Un changement de posture

On passe de « d'où vous connectez-vous ? » à « qui êtes-vous et quelle est votre intention ? ».

# Le flux d'accès Zero Trust




*Le Zero Trust n'est pas une vérification ponctuelle : chaque demande d'accès est validée en continu, en temps réel.*

# Identité des dispositifs & PKI : les fondations de la confiance

## Sans identité de dispositif

- N'importe quel appareil peut se connecter sans vérification
- Usurpation d'identité et attaques de type « man-in-the-middle »
- Accès non vérifié à travers tout le système
- Impossible de garantir l'origine réelle d'une donnée

## Avec identité de dispositif (PKI)

- Chaque dispositif s'authentifie, s'autorise et se protège de façon sécurisée
- Autorité de Certification (CA) émettant des certificats numériques uniques
- Communication chiffrée et vérifiée de bout en bout
- Sécurité par conception, dès la fabrication du dispositif

*On ne peut pas protéger ce que l'on ne peut pas identifier. On ne peut pas faire confiance à ce que l'on ne peut pas vérifier.*

# Le cycle de vie de l'identité d'un dispositif

**1. Fabrication & provisionnement**

Identité unique injectée dès la production du dispositif

**2. Émission du certificat**

Un certificat numérique est délivré par l'autorité de certification (CA)

**3. Enrôlement & authentification**

Le dispositif rejoint le réseau et s'authentifie en continu

**4. Supervision & renouvellement**

Surveillance continue, renouvellement du certificat avant expiration

**5. Révocation & mise au rebut**

Le certificat est révoqué à la fin de vie du dispositif

- Identité fiable, dès la fabrication jusqu'au retrait.
- Bonne identité, bon dispositif, au bon moment :
- c'est la fondation même du Zero Trust.

# Mises à jour OTA sécurisées (Over-The-Air)

## Sans mises à jour OTA sécurisées

- Vulnérabilités de sécurité non corrigées, risque élevé
- Maintenance manuelle, réparations physiques coûteuses et lentes
- Coûts opérationnels élevés à grande échelle
- Non-conformité réglementaire croissante

## Avec mises à jour OTA sécurisées

- Correctifs de sécurité déployés rapidement, à distance
- Mises à jour authentifiées et signées numériquement
- Coûts de maintenance réduits, agilité opérationnelle
- Conformité maintenue, confiance client renforcée

*Chaîne de confiance : dispositif de confiance → mise à jour de confiance → opération de confiance.*

# Cycle de vie sécurisé d'un dispositif IoT



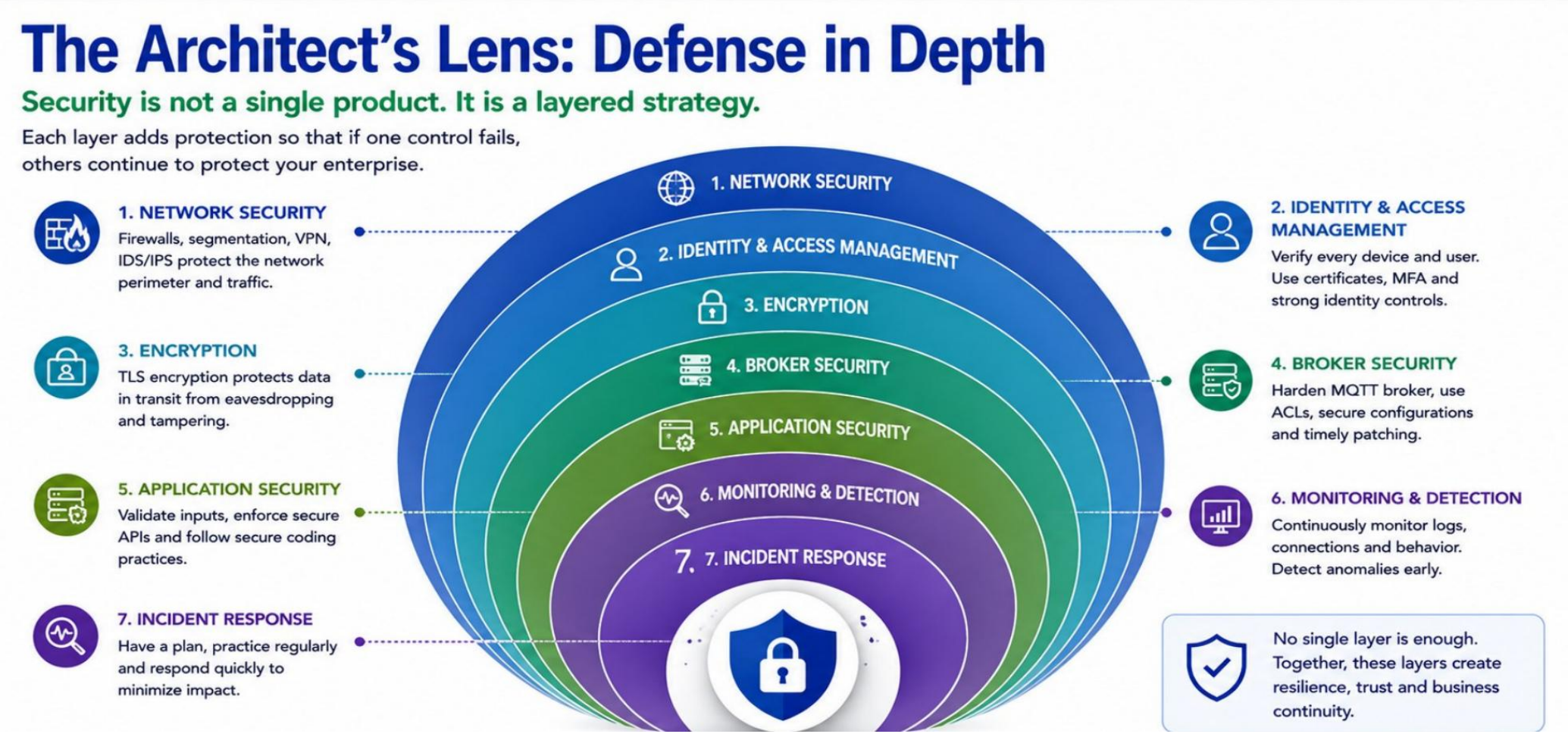

*La gestion du cycle de vie ne s'arrête jamais réellement : c'est une boucle continue de sécurité jusqu'au retrait du dispositif.*

# Panorama des normes et référentiels de sécurité IoT

| Norme / Référentiel | Portée | Apport principal |
|---|---|---|
| IEC 62443 | IoT industriel (OT/ICS) | Zones et conduits, niveaux de sécurité SL1 à SL4 |
| NIST Cybersecurity Framework | Gouvernance cybersécurité générale | Identify, Protect, Detect, Respond, Recover |
| ISO/IEC 27001 | Système de management de la sécurité de l'information | Gestion des risques, amélioration continue |
| ETSI EN 303 645 | IoT grand public | Exigences de sécurité de base pour dispositifs connectés |
| OWASP IoT Top 10 | Vulnérabilités applicatives IoT | Référentiel des risques de sécurité les plus courants |

*Aucun standard unique ne suffit : une approche en couches, combinant plusieurs référentiels, apporte une sécurité complète.*

# Défense en profondeur : 7 couches de protection




*Aucune couche seule ne suffit : ensemble, elles créent résilience, confiance et continuité d'activité.*

# 10 bonnes pratiques de sécurité pour l'IoT d'entreprise

1

### 1. Security by Design

Intégrer la sécurité dès la conception, jamais en dernière minute.

2

### 2. Zero Trust partout

Vérifier chaque identité, chaque connexion, sans exception.

3

### 3. Défense en profondeur

Superposer plusieurs couches de protection complémentaires.

4

### 4. Moindre privilège

Restreindre les accès au strict nécessaire, en continu.

5

### 5. Supervision continue

Détecter et répondre aux anomalies en temps réel.

6

### 6. Automatisation

Automatiser la conformité et la remédiation autant que possible.

7

### 7. Gestion du cycle de vie

Piloter chaque dispositif de la fabrication au retrait.

8

### 8. Culture & sensibilisation

Former les équipes ; la sécurité est l'affaire de tous.

9

### 9. Conformité by design

Aligner l'architecture avec les normes dès la conception.

10

### 10. Gestion du risque & résilience

Anticiper, planifier et pouvoir absorber les incidents.

MODULE 14

# Bibliographie & Références

Normes, spécifications techniques et documentation officielle utilisées dans ce cours

# Normes et référentiels de sécurité (Module 13)

| Référence | Organisme émetteur | Portée |
|---|---|---|
| IEC 62443 (série) | IEC / ISA | Sécurité des systèmes d'automatisation et de contrôle industriels (IACS) — zones, conduits, niveaux SL1-SL4 |
| NIST Cybersecurity Framework 2.0 | NIST (DOI 10.6028/NIST.CSWP.29, 2024) | Gouvernance cybersécurité : Govern, Identify, Protect, Detect, Respond, Recover |
| ISO/IEC 27001:2022 | ISO / IEC | Système de management de la sécurité de l'information (SMSI) |
| ETSI EN 303 645 V2.1.1 (2020) | ETSI (TC CYBER) | Référentiel de sécurité de base pour les dispositifs IoT grand public |
| OWASP Internet of Things Top 10 (2018) | OWASP Foundation | Les 10 risques de sécurité les plus critiques pour les systèmes IoT |

*Consultables sur : iec.ch · nist.gov/cyberframework · iso.org · etsi.org · owasp.org/www-project-internet-of-things-top-10*

# Spécifications techniques et protocoles (Modules 2-3)

| Spécification | Organisme | Utilisation dans le cours |
|---|---|---|
| MQTT Version 5.0 — OASIS Standard | OASIS / mqtt.org | Publish/Subscribe, QoS, topics, sécurité — Modules 3, TP1, TP3-5 |
| CoAP — RFC 7252 | IETF | Protocole applicatif léger pour dispositifs contraints — Module 2 |
| TLS 1.2 / 1.3 — RFC 5246 / RFC 8446 | IETF | Chiffrement des communications MQTT et REST — Modules 3, 6, 13 |
| HTTP/1.1 — RFC 9110 (2022) | IETF | Base des API REST — Module 3, TP8 |
| IEEE 802.15.4 (Zigbee) | IEEE | Réseaux personnels sans fil basse consommation — Module 2 |

*Spécifications consultables sur : mqtt.org · datatracker.ietf.org · ieee.org*

# Documentation officielle Cloud & plateformes (Modules 4, 6, 9)

| Plateforme / Outil | Éditeur | Documentation officielle |
|---|---|---|
| AWS IoT Core | Amazon Web Services | docs.aws.amazon.com/iot |
| Amazon DynamoDB | Amazon Web Services | docs.aws.amazon.com/dynamodb |
| Amazon Kinesis Data Streams | Amazon Web Services | docs.aws.amazon.com/kinesis |
| Eclipse Mosquitto | Eclipse Foundation | mosquitto.org |
| MongoDB | MongoDB Inc. | mongodb.com/docs |
| Docker | Docker Inc. | docs.docker.com |

# Bibliothèques et frameworks utilisés dans les TP

| Bibliothèque | Rôle | Documentation |
|---|---|---|
| Paho MQTT (Eclipse) | Client MQTT Python — TP1, TP3-5, TP8 | eclipse.dev/paho |
| PyMongo | Pilote MongoDB pour Python — TP7 | pymongo.readthedocs.io |
| FastAPI | Framework d'API REST Python — TP8 | fastapi.tiangolo.com |
| boto3 (AWS SDK for Python) | Interaction avec les services AWS — TP9, TP10 | boto3.amazonaws.com |
| scikit-learn | Machine Learning (Modules 7, TP11) | scikit-learn.org |
| TensorFlow Lite for Microcontrollers | Inférence Edge AI / TinyML — Module 8 | tensorflow.org/lite/microcontrollers |

# Organismes et fondations de référence

*Les organisations à l'origine des normes, protocoles et standards ouverts utilisés tout au long de ce cours.*

1. **NIST**

   National Institute of Standards and Technology (États-Unis) — nist.gov

2. **ISO / IEC**

   International Organization for Standardization / International Electrotechnical Commission — iso.org, iec.ch

3. **ETSI**

   European Telecommunications Standards Institute — etsi.org

4. **IETF**

   Internet Engineering Task Force (RFC) — ietf.org

5. **OASIS Open**

   Organisme de standardisation de la spécification MQTT — oasis-open.org

6. **Eclipse Foundation**

   Mosquitto, Paho MQTT et projets IoT open-source — eclipse.org

7. **OWASP Foundation**

   Open Web Application Security Project — owasp.org

8. **AWS (Amazon Web Services)**

   Documentation et architectures de référence Cloud IoT — aws.amazon.com

# Merci !

Passez maintenant à la fascicule de travaux pratiques pour mettre en œuvre chaque brique de cette architecture.